In [3]:

```
##############################################################################
##
# FINAL COMPLETE PRODUCTION-GRADE CODE
# Combines all features from both versions with fixes for robustness
# Covers: Feature engineering, Adversarial robustness, SHAP explanations,
# Latency tracking, Agentic SOC pipeline, Multiple feature subsets
##############################################################################
##

from __future__ import annotations
import os
import random
import time
import warnings
from contextlib import contextmanager
from dataclasses import dataclass, field
from typing import Dict, List, Any, Tuple, Optional
from collections import defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from matplotlib.patches import Patch

# Suppress warnings
warnings.filterwarnings('ignore')

# ============================================================
# PROFESSIONAL PLOTTING CONFIGURATION
# ============================================================
matplotlib.rcParams['figure.figsize'] = (10, 6)
matplotlib.rcParams['font.size'] = 12
matplotlib.rcParams['axes.labelsize'] = 14
matplotlib.rcParams['axes.titlesize'] = 16
matplotlib.rcParams['xtick.labelsize'] = 12
matplotlib.rcParams['ytick.labelsize'] = 12
matplotlib.rcParams['legend.fontsize'] = 12
matplotlib.rcParams['figure.titlesize'] = 18
matplotlib.rcParams['font.family'] = 'sans-serif'
matplotlib.rcParams['font.sans-serif'] = ['Arial', 'DejaVu Sans']
matplotlib.rcParams['axes.linewidth'] = 1.5
matplotlib.rcParams['xtick.major.width'] = 1.5
matplotlib.rcParams['ytick.major.width'] = 1.5
matplotlib.rcParams['xtick.major.size'] = 6
matplotlib.rcParams['ytick.major.size'] = 6
matplotlib.rcParams['savefig.dpi'] = 300
matplotlib.rcParams['savefig.bbox'] = 'tight'
matplotlib.rcParams['savefig.pad_inches'] = 0.1
```

```
COLORS = {
    'primary':   '#2E86AB',    # Blue  (RF)
    'secondary': '#A23B72',    # Purple (IF)
    'success':   '#28A745',    # Green
    'warning':   '#F18F01',    # Orange
    'danger':    '#C73E1D',    # Red
    'neutral':   '#6C757D',    # Gray
    'light':     '#E9ECEF',    # Light Gray
    'dark':      '#343A40',    # Dark Gray
    'xgb':       '#F7931E',    # XGBoost orange
}

from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.metrics import (
    classification_report, confusion_matrix, precision_score, recall_score,
    f1_score, accuracy_score, roc_curve, auc, precision_recall_curve,
    average_precision_score,
)
from sklearn.inspection import permutation_importance
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
import xgboost as xgb
import shap

# ============================================================
# CONFIG
# ============================================================
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)

DATASET_PATHS = [
    "/content/Benign-Monday-no-metadata.parquet",
    "/content/Botnet-Friday-no-metadata.parquet",
    "/content/Bruteforce-Tuesday-no-metadata.parquet",
    "/content/DDoS-Friday-no-metadata.parquet",
    "/content/DoS-Wednesday-no-metadata.parquet",
    "/content/Infiltration-Thursday-no-metadata.parquet",
    "/content/Portscan-Friday-no-metadata.parquet",
    "/content/WebAttacks-Thursday-no-metadata.parquet",
]

# ✅ BASELINE 5 FEATURES (reference point)
FEATURE_COLUMNS = [
    "Flow Duration",
    "Total Fwd Packets",
    "Total Backward Packets",
    "Flow Bytes/s",
    "Flow Packets/s",
```

```
]

# ✅ EXTENDED FEATURE POOL — for subset experiments
ALL_POSSIBLE_FEATURES = [
    "Flow Duration",
    "Total Fwd Packets",
    "Total Backward Packets",
    "Flow Bytes/s",
    "Flow Packets/s",
    "Fwd Packet Length Max",
    "Fwd Packet Length Min",
    "Fwd Packet Length Mean",
    "Fwd Packet Length Std",
    "Bwd Packet Length Max",
    "Bwd Packet Length Min",
    "Bwd Packet Length Mean",
    "Bwd Packet Length Std",
    "Flow IAT Mean",
    "Flow IAT Std",
    "Flow IAT Max",
    "Flow IAT Min",
    "Fwd IAT Total",
    "Fwd IAT Mean",
    "Fwd IAT Std",
    "Fwd Packets/s",
    "Bwd Packets/s",
    "Packet Length Min",
    "Packet Length Max",
    "Packet Length Mean",
    "Packet Length Std",
    "Packet Length Variance",
    "FIN Flag Count",
    "SYN Flag Count",
    "RST Flag Count",
    "PSH Flag Count",
    "ACK Flag Count",
    "Down/Up Ratio",
    "Average Packet Size",
    "Avg Fwd Segment Size",
    "Avg Bwd Segment Size",
    "Subflow Fwd Packets",
    "Subflow Fwd Bytes",
    "Subflow Bwd Packets",
    "Subflow Bwd Bytes",
    "Init_Win_bytes_forward",
    "Init_Win_bytes_backward",
    "Active Mean",
    "Active Std",
    "Idle Mean",
    "Idle Std",
]

LABEL_COLUMN = "Label"
```

```
# ============================================================
# LATENCY TRACKER — Global instrumentation
# ============================================================
class LatencyTracker:
    """Measures and reports wall-clock latency for named operations."""

    def __init__(self):
        self.records: Dict[str, List[float]] = defaultdict(list)

    @contextmanager
    def measure(self, label: str):
        start = time.perf_counter()
        yield
        self.records[label].append(time.perf_counter() - start)

    def report(self):
        print_section("LATENCY REPORT")
        rows = self.summary_rows()
        if not rows:
            print("  No measurements recorded.")
            return
        df = pd.DataFrame(rows)
        print(df.to_string(index=False))

    def summary_rows(self) -> List[Dict]:
        rows = []
        for label, times in self.records.items():
            rows.append({
                "Operation":  label,
                "Mean (ms)":  round(np.mean(times) * 1000, 3),
                "Std (ms)":   round(np.std(times) * 1000, 3),
                "Min (ms)":   round(np.min(times) * 1000, 3),
                "Max (ms)":   round(np.max(times) * 1000, 3),
                "Total (s)":  round(np.sum(times), 4),
                "N":          len(times),
            })
        return rows

    def plot(self, save_path=None):
        rows = self.summary_rows()
        if not rows:
            return
        df = pd.DataFrame(rows)
        fig, ax = plt.subplots(figsize=(12, max(4, len(df) * 0.55)))
        colors = [COLORS['primary'] if 'Train' in r['Operation']
                  else COLORS['xgb'] if 'XGB' in r['Operation']
                  else COLORS['secondary'] if 'Inference' in r['Operation']
                  else COLORS['neutral'] for r in rows]
        bars = ax.barh(df['Operation'], df['Mean (ms)'],
                       xerr=df['Std (ms)'], capsize=4,
                       color=colors, edgecolor='black', linewidth=1.2,
alpha=0.9)
```

```
        ax.set_xlabel('Mean Latency (ms)', fontsize=13, fontweight='bold')
        ax.set_title('Operation Latency Profile', fontsize=15,
fontweight='bold', pad=12)
        ax.invert_yaxis()
        ax.grid(axis='x', linestyle='--', alpha=0.35)
        for bar in bars:
            w = bar.get_width()
            ax.annotate(f'{w:.2f}ms',
                        xy=(w, bar.get_y() + bar.get_height() / 2),
                        xytext=(4, 0), textcoords='offset points',
                        va='center', fontsize=10, fontweight='bold')
        plt.tight_layout()
        if save_path:
            plt.savefig(save_path, dpi=300, bbox_inches='tight')
        plt.show()
        return fig

# Global latency tracker
LATENCY = LatencyTracker()

# ============================================================
# UTILITIES
# ============================================================
def print_section(title: str):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)

def normalize_label(label):
    label = str(label).strip()
    if label.upper() in {"BENIGN", "NORMAL"}:
        return "BENIGN"
    return label

def label_to_binary(label):
    return 0 if normalize_label(label) == "BENIGN" else 1

def safe_float(x, default=0.0):
    try:
        v = float(x)
        return v if np.isfinite(v) else default
    except Exception:
        return default

def score_to_confidence(score: float) -> float:
    return float(1 / (1 + np.exp(-np.clip(score, -50, 50))))

# ============================================================
# DATA LOADING & PREPROCESSING
# ============================================================
def load_data(paths):
    """Load and concatenate all parquet files."""
    dfs = []
```

```
    for p in paths:
        if os.path.exists(p):
            print("Loading:", p)
            dfs.append(pd.read_parquet(p))
        else:
            print("Skipping missing file:", p)
    if not dfs:
        raise FileNotFoundError("No parquet dataset files found. Check
DATASET_PATHS.")
    return pd.concat(dfs, ignore_index=True)

def preprocess(df, sample_size=500000, feature_cols=None):
    """
    Preprocess dataframe. Uses only available features (no zero-filling).
    ✅ FIXED: No longer fills missing columns with 0
    """
    if feature_cols is None:
        feature_cols = FEATURE_COLUMNS

    # Keep only features that exist in dataframe
    available_features = [f for f in feature_cols if f in df.columns]
    if not available_features:
        raise ValueError(f"No available features found. Dataset columns:
{df.columns.tolist()}")

    needed = available_features + [LABEL_COLUMN]
    df = df[needed].copy()

    # Convert features to numeric, drop NaN
    for c in available_features:
        df[c] = pd.to_numeric(df[c], errors="coerce")

    # Normalize labels
    df[LABEL_COLUMN] = df[LABEL_COLUMN].astype(str).map(normalize_label)

    # Replace inf with NaN, then drop all NaN rows
    df = df.replace([np.inf, -np.inf], np.nan).dropna()

    if df.empty:
        raise ValueError("Dataset became empty after preprocessing.")

    # Sample if necessary (upper bound only)
    if len(df) > sample_size:
        df = df.sample(sample_size, random_state=RANDOM_SEED)

    return df.reset_index(drop=True), available_features

def get_available_features(df, candidate_list):
    """Return only features present in dataframe."""
    return [f for f in candidate_list if f in df.columns]

def df_to_logs(df, feature_cols=None):
    """Convert dataframe to network logs for agentic SOC pipeline."""
```

```
    if feature_cols is None:
        feature_cols = FEATURE_COLUMNS

    logs = []
    src_pool = [f"10.0.{i//256}.{i%256}" for i in range(1, 100)]
    dst_pool = [f"192.168.{i//256}.{i%256}" for i in range(1, 50)]
    port_pool = [80, 443, 22, 21, 53, 8080, 3389, 25]

    for i, r in df.iterrows():
        log = {
            "label":     str(r[LABEL_COLUMN]),
            "src_ip":    src_pool[i % len(src_pool)],
            "dst_ip":    dst_pool[i % len(dst_pool)],
            "port":      port_pool[i % len(port_pool)],
            "timestamp": i,
        }
        # Add all available features
        for fc in feature_cols:
            if fc in r.index:
                log[fc] = safe_float(r[fc])
        logs.append(log)
    return logs

def make_single_row_df(log: Dict[str, Any], feature_names: List[str]) ->
pd.DataFrame:
    """Convert single log entry to DataFrame for inference."""
    row_data = [safe_float(log.get(feat, 0.0)) for feat in feature_names]
    return pd.DataFrame([row_data], columns=feature_names)

# ============================================================
# RESULT TRACKER FOR AGENTIC SOC
# ============================================================
class ResultTracker:
    """Tracks all agentic pipeline decisions and metrics."""

    def __init__(self, name: str):
        self.name = name
        self.total_logs = 0
        self.anomalies = 0
        self.incidents = 0
        self.severity_count = defaultdict(int)
        self.auto_actions = 0
        self.hitl_actions = 0
        self.novelty_scores = []
        self.confidence_scores = []
        self.incident_tp = 0
        self.incident_fp = 0
        self.decisions_with_details = []
        self.hitl_sample_rows: List[Dict] = []

    def log_analysis(self, alerts):
        self.total_logs += len(alerts)
        self.anomalies += sum(int(a["anomaly"]) for a in alerts)
```

```
    def log_incidents(self, incidents):
        self.incidents += len(incidents)

    def log_enrichment(self, severity):
        self.severity_count[severity] += 1

    def log_decision(self, decision, incident_members=None):
        if decision["type"] == "AUTO":
            self.auto_actions += 1
        else:
            self.hitl_actions += 1
            if incident_members:
                for m in incident_members:
                    self.hitl_sample_rows.append(m["log"])
        self.novelty_scores.append(float(decision["novelty"]))
        self.confidence_scores.append(float(decision["confidence"]))
        self.decisions_with_details.append(decision)

    def log_incident_accuracy(self, is_true_positive):
        if is_true_positive:
            self.incident_tp += 1
        else:
            self.incident_fp += 1

    def summary_dict(self):
        total_decisions = self.auto_actions + self.hitl_actions
        incident_precision = (
            self.incident_tp / (self.incident_tp + self.incident_fp)
            if (self.incident_tp + self.incident_fp) > 0 else 0.0
        )
        return {
            "Pipeline":         self.name,
            "Logs":             self.total_logs,
            "Anomalies":        self.anomalies,
            "Incidents":        self.incidents,
            "AUTO":             self.auto_actions,
            "HITL":             self.hitl_actions,
            "AUTO Rate":        (self.auto_actions / total_decisions) if
total_decisions else 0.0,
            "HITL Rate":        (self.hitl_actions / total_decisions) if
total_decisions else 0.0,
            "Incident Precision": incident_precision,
            "Avg Novelty":      float(np.mean(self.novelty_scores))    if
self.novelty_scores else 0.0,
            "Max Novelty":      float(np.max(self.novelty_scores))     if
self.novelty_scores else 0.0,
            "Avg Confidence":   float(np.mean(self.confidence_scores)) if
self.confidence_scores else 0.0,
        }

    def severity_dict(self):
        return dict(self.severity_count)
```

```
    def report(self):
        print_section(f"AGENTIC SOC RESULTS — {self.name}")
        print(f"Total Logs     : {self.total_logs}")
        print(f"Anomalies      : {self.anomalies}")
        print(f"Incidents      : {self.incidents}")
        print(f"AUTO Decisions : {self.auto_actions}")
        print(f"HITL Decisions : {self.hitl_actions}")
        print(f"Incident TP    : {self.incident_tp}")
        print(f"Incident FP    : {self.incident_fp}")
        print("\nSeverity Distribution:")
        if self.severity_count:
            for k, v in sorted(self.severity_count.items()):
                print(f"  {k:<10} : {v}")
        else:
            print("  None")
        if self.novelty_scores:
            print("\nNovelty Summary:")
            print(f"  Avg Novelty  : {np.mean(self.novelty_scores):.4f}")
            print(f"  Max Novelty  : {np.max(self.novelty_scores):.4f}")
            print(f"  Min Novelty  : {np.min(self.novelty_scores):.4f}")
        if self.confidence_scores:
            print("\nConfidence Summary:")
            print(f"  Avg Confidence:
{np.mean(self.confidence_scores):.4f}")
            print(f"  Max Confidence:
{np.max(self.confidence_scores):.4f}")
            print(f"  Min Confidence:
{np.min(self.confidence_scores):.4f}")
        print(f"\nHITL samples for SHAP: {len(self.hitl_sample_rows)}")

    def plot(self, prefix: str = ""):
        """Generate diagnostic plots."""
        title_prefix = f"{prefix} " if prefix else ""

        if self.severity_count:
            fig, ax = plt.subplots(figsize=(10, 6))
            severity_order = ["low", "medium", "high", "critical"]
            labels = [s for s in severity_order if s in
self.severity_count]
            values = [self.severity_count[s] for s in labels]
            color_map = {
                "low": COLORS["success"],
                "medium": COLORS["warning"],
                "high": COLORS["danger"],
                "critical": COLORS["dark"],
            }
            bar_colors = [color_map[l] for l in labels]
            bars = ax.bar(labels, values, color=bar_colors,
edgecolor='black', linewidth=1.5, alpha=0.9)
            ax.set_title(f"{title_prefix}Severity Distribution",
fontsize=14, fontweight='bold', pad=15)
            ax.set_xlabel("Severity Level", fontsize=12, fontweight='bold')
```

```
            ax.set_ylabel("Count", fontsize=12, fontweight='bold')
            ax.grid(axis="y", linestyle="--", alpha=0.3)
            for bar in bars:
                height = bar.get_height()
                ax.text(bar.get_x() + bar.get_width() / 2.0, height, f"
{int(height)}",
                        ha="center", va="bottom", fontweight='bold',
fontsize=10)
            plt.tight_layout()
            plt.show()

        fig, ax = plt.subplots(figsize=(10, 6))
        bars = ax.bar(["AUTO", "HITL"], [self.auto_actions,
self.hitl_actions],
                      color=[COLORS["success"], COLORS["danger"]],
                      edgecolor='black', linewidth=1.5, alpha=0.9)
        ax.set_title(f"{title_prefix}Escalation Decisions", fontsize=14,
fontweight='bold', pad=15)
        ax.set_ylabel("Count", fontsize=12, fontweight='bold')
        ax.grid(axis="y", linestyle="--", alpha=0.3)
        for bar in bars:
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width() / 2.0, height, f"
{int(height)}",
                    ha="center", va="bottom", fontweight='bold',
fontsize=11)
        plt.tight_layout()
        plt.show()

        if self.novelty_scores:
            fig, ax = plt.subplots(figsize=(10, 6))
            ax.hist(self.novelty_scores, bins=15, color=COLORS["primary"],
                    edgecolor='black', linewidth=1.5, alpha=0.8)
            ax.set_title(f"{title_prefix}Novelty Scores Distribution",
fontsize=14, fontweight='bold', pad=15)
            ax.set_xlabel("Novelty Score", fontsize=12, fontweight='bold')
            ax.set_ylabel("Frequency", fontsize=12, fontweight='bold')
            ax.axvline(np.mean(self.novelty_scores), color='red',
linestyle='--', linewidth=2.5,
                       label=f"Mean: {np.mean(self.novelty_scores):.3f}")
            ax.axvline(0.95, color=COLORS["danger"], linestyle=':',
linewidth=2.5, label="Threshold (0.95)")
            ax.legend(fontsize=10, loc="upper right")
            ax.grid(axis="y", linestyle="--", alpha=0.3)
            plt.tight_layout()
            plt.show()

# ============================================================
# MODEL BUILDING — RF, IF, XGBoost with latency tracking
# ============================================================
def build_iforest(X_train: pd.DataFrame):
    """Build Isolation Forest with scaling."""
    model = Pipeline([
```

```
        ("scaler", StandardScaler()),
        ("iforest", IsolationForest(
            n_estimators=200,
            contamination=0.15,
            random_state=RANDOM_SEED,
        )),
    ])
    with LATENCY.measure("IF Training"):
        model.fit(X_train)
    return model

def build_rf_smote(X_train: pd.DataFrame, y_train: pd.Series):
    """Build Random Forest with SMOTE oversampling."""
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_train)
    smote = SMOTE(random_state=RANDOM_SEED)
    X_res, y_res = smote.fit_resample(X_scaled, y_train)

    model = RandomForestClassifier(
        n_estimators=100,
        max_depth=8,
        min_samples_split=10,
        min_samples_leaf=5,
        random_state=RANDOM_SEED,
        n_jobs=-1,
    )
    with LATENCY.measure("RF Training"):
        model.fit(X_res, y_res)
    return model, scaler

def build_xgboost(X_train: pd.DataFrame, y_train: pd.Series):
    """Build XGBoost with SMOTE oversampling."""
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_train)
    smote = SMOTE(random_state=RANDOM_SEED)
    X_res, y_res = smote.fit_resample(X_scaled, y_train)

    scale_pos_weight = float((y_res == 0).sum()) / float((y_res == 1).sum()
+ 1e-9)
    model = xgb.XGBClassifier(
        n_estimators=100,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        scale_pos_weight=scale_pos_weight,
        random_state=RANDOM_SEED,
        eval_metric='logloss',
        verbosity=0,
        n_jobs=-1,
    )
    with LATENCY.measure("XGB Training"):
        model.fit(X_res, y_res)
```

```
    return model, scaler

# ============================================================
# CROSS-VALIDATION
# ============================================================
def perform_cross_validation(X, y, model_name, model_estimator,
n_splits=3):
    """Perform stratified k-fold cross-validation."""
    print_section(f"CROSS-VALIDATION — {model_name}")
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
random_state=RANDOM_SEED)
    try:
        scores = cross_val_score(model_estimator, X, y, cv=skf,
                                 scoring='f1', n_jobs=-1)
        print(f"{model_name} CV F1 Scores : {scores}")
        print(f"{model_name} CV Mean F1   : {scores.mean():.4f} (+/-
{scores.std()*2:.4f})")
        return scores
    except Exception as e:
        print(f"Error during CV for {model_name}: {e}")
        return np.array([])

# ============================================================
# AGENTS FOR AGENTIC SOC PIPELINE
# ============================================================
class LogAnalysisAgent:
    """Analyzes individual logs for anomalies."""

    def __init__(self, model, model_type, scaler=None, feature_names=None):
        self.model = model
        self.model_type = model_type
        self.scaler = scaler
        self.feature_names = feature_names or FEATURE_COLUMNS

    def analyze(self, log):
        x_df = make_single_row_df(log, self.feature_names)
        with LATENCY.measure(f"Inference ({self.model_type})"):
            if self.model_type == "iforest":
                normality  = float(self.model.decision_function(x_df)[0])
                score      = float(-normality)
                confidence = score_to_confidence(normality)
                anomaly    = bool(self.model.predict(x_df)[0] == -1)
            elif self.model_type in ("rf", "xgb"):
                x_scaled   = self.scaler.transform(x_df)
                prob       = float(self.model.predict_proba(x_scaled)[0]
[1])
                score      = prob
                confidence = prob
                anomaly    = bool(prob > 0.5)
            else:
                raise ValueError(f"Unknown model_type: {self.model_type}")
        return {
            "log": log, "anomaly": anomaly,
```

```
            "confidence": confidence, "score": score, "anomaly_score":
score,
        }

class CorrelationAgent:
    """Correlates individual alerts into incidents."""

    def correlate(self, alerts):
        bucket = defaultdict(list)
        for a in alerts:
            if not a["anomaly"]:
                continue
            log = a["log"]
            risk_bucket = "high" if a["score"] >= 0.5 else "medium"
            key = (log["src_ip"], log["dst_ip"], log["port"], risk_bucket)
            bucket[key].append(a)

        incidents = []
        for (src, dst, port, risk_bucket), members in bucket.items():
            incidents.append({
                "incident_id": f"
{src}_{dst}_{port}_{risk_bucket}_{len(members)}",
                "src_ip": src, "dst_ip": dst, "port": port,
                "risk_bucket": risk_bucket,
                "count": len(members), "members": members,
            })
        return incidents

class ThreatEnrichmentAgent:
    """Enriches incidents with severity labels."""

    def enrich(self, inc):
        c         = inc["count"]
        avg_score = np.mean([m["score"] for m in inc["members"]]) if
inc["members"] else 0.0

        if   c >= 20 and avg_score >= 0.8: severity = "critical"
        elif c >= 10 or  avg_score >= 0.9: severity = "high"
        elif c >= 5:                       severity = "medium"
        else:                              severity = "low"
        return {"incident": inc, "severity": severity}

@dataclass
class SafetyBoundary:
    """Decision thresholds for AUTO vs HITL escalation."""
    confidence_threshold: float = 0.80
    novelty_threshold:    float = 0.95

    def check(self, confidence, severity, novelty):
        if severity == "critical":              return "ESCALATE"
        if confidence < self.confidence_threshold: return "ESCALATE"
        if novelty >= self.novelty_threshold:   return "ESCALATE"
        return "AUTO"
```

```
class EscalationAgent:
    """Makes AUTO vs HITL decisions."""

    def __init__(self):
        self.bound = SafetyBoundary()

    def novelty_score(self, inc):
        score = 0.0
        if inc["count"] == 1:
            score += 0.3
        avg_score = float(np.mean([m["score"] for m in inc["members"]])) if
inc["members"] else 0.0
        if avg_score > 0.7:
            score += 0.4
        if inc["risk_bucket"] == "high":
            score += 0.3
        return float(min(score, 1.0))

    def decide(self, enriched):
        inc      = enriched["incident"]
        conf     = float(np.mean([m["confidence"] for m in
inc["members"]])) if inc["members"] else 0.0
        severity = enriched["severity"]
        novelty  = self.novelty_score(inc)
        decision = self.bound.check(conf, severity, novelty)
        return {
            "type":          "AUTO" if decision == "AUTO" else "HITL",
            "severity":      severity,
            "confidence":    conf,
            "novelty":       novelty,
            "decision_rule": decision,
            "incident_id":   inc["incident_id"],
            "risk_bucket":   inc["risk_bucket"],
            "count":         inc["count"],
        }

# ============================================================
# ORCHESTRATOR — Runs full agentic pipeline
# ============================================================
class Orchestrator:
    """Orchestrates the full agentic SOC pipeline."""

    def __init__(self, model, model_type, scaler=None, name="Pipeline",
feature_names=None):
        self.name      = name
        self.log_agent = LogAnalysisAgent(model, model_type, scaler,
feature_names)
        self.corr      = CorrelationAgent()
        self.enrich_ag = ThreatEnrichmentAgent()
        self.esc       = EscalationAgent()

    def run(self, logs):
```

```
        tracker = ResultTracker(self.name)

        # Stage 1: Log Analysis
        alerts  = [self.log_agent.analyze(l) for l in logs]
        tracker.log_analysis(alerts)

        # Stage 2: Correlation
        incidents = self.corr.correlate(alerts)
        tracker.log_incidents(incidents)

        # Stage 3-5: Enrichment, Escalation, Accuracy Check
        decisions = []
        for i in incidents:
            e = self.enrich_ag.enrich(i)
            tracker.log_enrichment(e["severity"])
            d = self.esc.decide(e)

            # Check incident accuracy
            actual_labels = [m["log"]["label"] for m in i["members"]]
            attack_count  = sum(1 for l in actual_labels if
normalize_label(l) != "BENIGN")
            is_tp = (attack_count / len(actual_labels)) > 0.5
            tracker.log_incident_accuracy(is_tp)

            # Log decision with HITL sample storage
            tracker.log_decision(d, incident_members=i["members"] if
d["type"] == "HITL" else None)
            decisions.append({**e, **d})

        return {"alerts": alerts, "incidents": incidents,
                "decisions": decisions, "tracker": tracker}

# ============================================================
# MODEL EVALUATION & METRICS
# ============================================================
def print_metrics(name, y_true, y_pred):
    """Print comprehensive evaluation metrics."""
    acc  = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec  = recall_score(y_true, y_pred, zero_division=0)
    f1   = f1_score(y_true, y_pred, zero_division=0)
    cm   = confusion_matrix(y_true, y_pred)

    print_section(f"{name} RESULTS")
    print(f"Accuracy : {acc:.6f}")
    print(f"Precision: {prec:.6f}")
    print(f"Recall   : {rec:.6f}")
    print(f"F1-score : {f1:.6f}")
    print("\nConfusion Matrix:")
    print(cm)
    print("\nClassification Report:")
    print(classification_report(y_true, y_pred, zero_division=0))
```

```
    return {
        "Model": name, "Accuracy": acc, "Precision": prec,
        "Recall": rec, "F1": f1, "Confusion Matrix": cm,
    }

# ============================================================
# FEATURE SUBSET EXPERIMENT
# ============================================================
def run_feature_subset_experiment(df_full, all_features, save_path=None):
    """
    Compare models across 3 feature subsets:
    - Baseline-5: Original 5 features
    - Top-10: Top 10 by importance
    - Top-20: Top 20 by importance
    ✅ FIXED: Uses actual available features, no zero-filling
    """
    print_section("FEATURE SUBSET EXPERIMENT: 5 vs TOP-10 vs TOP-20")

    y_full = df_full[LABEL_COLUMN].map(label_to_binary)
    available = get_available_features(df_full, ALL_POSSIBLE_FEATURES)
    print(f"  Features available in dataset : {len(available)}")

    if len(available) < 6:
        print("  Not enough features found. Skipping subset experiment.")
        return {}

    # Train RF on all features to get importance
    X_full = df_full[available].copy()
    for c in available:
        X_full[c] = pd.to_numeric(X_full[c], errors='coerce').fillna(0)
    X_full = X_full.replace([np.inf, -np.inf], 0)

    X_tr, X_te, y_tr, y_te = train_test_split(
        X_full, y_full, test_size=0.3,
        stratify=y_full, random_state=RANDOM_SEED)

    scaler_full = StandardScaler()
    X_tr_sc = scaler_full.fit_transform(X_tr)
    X_te_sc = scaler_full.transform(X_te)
    smote   = SMOTE(random_state=RANDOM_SEED)
    X_res, y_res = smote.fit_resample(X_tr_sc, y_tr)

    rf_full = RandomForestClassifier(
        n_estimators=100, max_depth=8,
        min_samples_split=10, min_samples_leaf=5,
        random_state=RANDOM_SEED, n_jobs=-1)
    rf_full.fit(X_res, y_res)

    importances = rf_full.feature_importances_
    sorted_idx  = np.argsort(importances)[::-1]
    sorted_feats = [available[i] for i in sorted_idx]

    print("\n  Top-20 features by Gini importance:")
```

```
    for rank, (feat, imp) in enumerate(zip(sorted_feats[:20],
                                           importances[sorted_idx[:20]]),
1):
        print(f"    {rank:2d}. {feat:<45}  {imp:.6f}")

    # Build subsets
    top10 = sorted_feats[:min(10, len(sorted_feats))]
    top20 = sorted_feats[:min(20, len(sorted_feats))]
    subsets = {
        "Baseline-5": FEATURE_COLUMNS,
        "Top-10":     top10,
        "Top-20":     top20,
    }

    results = {}
    for subset_name, feats in subsets.items():
        use_feats = [f for f in feats if f in df_full.columns]
        print(f"\n  ── {subset_name} ({len(use_feats)} features) ──")

        Xsub = df_full[use_feats].copy()
        for c in use_feats:
            Xsub[c] = pd.to_numeric(Xsub[c], errors='coerce').fillna(0)
        Xsub = Xsub.replace([np.inf, -np.inf], 0)
        ysub = y_full

        Xtr, Xte, ytr, yte = train_test_split(
            Xsub, ysub, test_size=0.3,
            stratify=ysub, random_state=RANDOM_SEED)

        sc_sub  = StandardScaler()
        Xtr_sc  = sc_sub.fit_transform(Xtr)
        Xte_sc  = sc_sub.transform(Xte)
        sm_sub  = SMOTE(random_state=RANDOM_SEED)
        Xr, yr  = sm_sub.fit_resample(Xtr_sc, ytr)

        # RF
        t0 = time.perf_counter()
        rf_sub = RandomForestClassifier(n_estimators=100, max_depth=8,
                                        min_samples_split=10,
min_samples_leaf=5,
                                        random_state=RANDOM_SEED,
n_jobs=-1)
        rf_sub.fit(Xr, yr)
        rf_train_t = time.perf_counter() - t0

        t0 = time.perf_counter()
        yp_rf = rf_sub.predict(Xte_sc)
        rf_inf_t = (time.perf_counter() - t0) / len(Xte_sc) * 1000

        rf_f1  = f1_score(yte, yp_rf, zero_division=0)
        rf_acc = accuracy_score(yte, yp_rf)

        # XGBoost
```

```
        t0 = time.perf_counter()
        xgb_sub = xgb.XGBClassifier(n_estimators=100, max_depth=6,
                                    learning_rate=0.1, subsample=0.8,
                                    colsample_bytree=0.8,
random_state=RANDOM_SEED,
                                    eval_metric='logloss', verbosity=0,
n_jobs=-1)
        xgb_sub.fit(Xr, yr)
        xgb_train_t = time.perf_counter() - t0

        t0 = time.perf_counter()
        yp_xgb  = xgb_sub.predict(Xte_sc)
        xgb_inf_t = (time.perf_counter() - t0) / len(Xte_sc) * 1000

        xgb_f1  = f1_score(yte, yp_xgb, zero_division=0)
        xgb_acc = accuracy_score(yte, yp_xgb)

        print(f"    RF  — F1={rf_f1:.4f}  Acc={rf_acc:.4f}  TrainT=
{rf_train_t:.2f}s  InfT={rf_inf_t:.4f}ms")
        print(f"    XGB — F1={xgb_f1:.4f}  Acc={xgb_acc:.4f}  TrainT=
{xgb_train_t:.2f}s  InfT={xgb_inf_t:.4f}ms")

        results[subset_name] = {
            "n_features": len(use_feats),
            "RF_F1": rf_f1, "RF_Acc": rf_acc, "RF_TrainTime_s": rf_train_t,
            "RF_InfTime_ms_per_sample": rf_inf_t,
            "XGB_F1": xgb_f1, "XGB_Acc": xgb_acc, "XGB_TrainTime_s":
xgb_train_t,
            "XGB_InfTime_ms_per_sample": xgb_inf_t,
        }

    # Print summary
    print_section("FEATURE SUBSET EXPERIMENT — SUMMARY TABLE")
    rows = []
    for sname, r in results.items():
        rows.append({
            "Subset":  sname, "N_feat": r["n_features"],
            "RF_F1":   round(r["RF_F1"],  4), "RF_Acc":  round(r["RF_Acc"],
4),
            "XGB_F1":  round(r["XGB_F1"], 4), "XGB_Acc":
round(r["XGB_Acc"], 4),
        })
    print(pd.DataFrame(rows).to_string(index=False))

    # Plot
    if results:
        subset_names = list(results.keys())
        rf_f1s  = [results[s]["RF_F1"]  for s in subset_names]
        xgb_f1s = [results[s]["XGB_F1"] for s in subset_names]
        x = np.arange(len(subset_names))
        width = 0.35

        fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```
        # F1 comparison
        axes[0].bar(x - width/2, rf_f1s,  width, label='RF+SMOTE',
                    color=COLORS['primary'],   edgecolor='black',
linewidth=1.4, alpha=0.9)
        axes[0].bar(x + width/2, xgb_f1s, width, label='XGBoost',
                    color=COLORS['xgb'],       edgecolor='black',
linewidth=1.4, alpha=0.9)
        axes[0].set_xticks(x)
        axes[0].set_xticklabels(subset_names, fontsize=12)
        axes[0].set_ylabel('F1 Score', fontsize=13, fontweight='bold')
        axes[0].set_title('F1 Score vs Feature Subset', fontsize=14,
fontweight='bold', pad=10)
        axes[0].set_ylim(0, 1.05)
        axes[0].legend(fontsize=11); axes[0].grid(axis='y', linestyle='--',
alpha=0.35)

        # Latency comparison
        rf_train_ts  = [results[s]["RF_TrainTime_s"]  for s in
subset_names]
        xgb_train_ts = [results[s]["XGB_TrainTime_s"] for s in
subset_names]
        axes[1].bar(x - width/2, rf_train_ts,  width, label='RF+SMOTE',
                    color=COLORS['primary'], edgecolor='black',
linewidth=1.4, alpha=0.9)
        axes[1].bar(x + width/2, xgb_train_ts, width, label='XGBoost',
                    color=COLORS['xgb'],    edgecolor='black',
linewidth=1.4, alpha=0.9)
        axes[1].set_xticks(x)
        axes[1].set_xticklabels(subset_names, fontsize=12)
        axes[1].set_ylabel('Training Time (s)', fontsize=13,
fontweight='bold')
        axes[1].set_title('Training Time vs Feature Subset', fontsize=14,
fontweight='bold', pad=10)
        axes[1].legend(fontsize=11); axes[1].grid(axis='y', linestyle='--',
alpha=0.35)

        plt.suptitle('Feature Subset Experiment: Performance vs
Complexity',
                     fontsize=16, fontweight='bold', y=1.02)
        plt.tight_layout()
        if save_path:
            plt.savefig(save_path, dpi=300, bbox_inches='tight')
        plt.show()

    return results

# ============================================================
# GAUSSIAN NOISE PERTURBATION TEST — Robustness
# ============================================================
def gaussian_noise_perturbation_test(rf_model, rf_scaler,
                                     xgb_model, xgb_scaler,
                                     X_test: pd.DataFrame,
                                     y_test: pd.Series,
```

```
                                        feature_names: List[str],
                                        noise_levels: Tuple = (0.0, 0.01,
0.05,
                                                               0.10, 0.25,
0.50, 1.0, 2.0),
                                        save_path: str = None):
    """
    Add zero-mean Gaussian noise with std = noise_level × feature_std.
    Measure F1 degradation under noise to assess real-world robustness.
    """
    print_section("GAUSSIAN NOISE PERTURBATION TEST — F1 DEGRADATION")

    X_arr = X_test.values.astype(float)
    y_arr = y_test.values
    feat_stds = X_arr.std(axis=0) + 1e-9
    N_REPEATS = 5

    # Baseline
    rf_base_f1  = f1_score(y_arr,
rf_model.predict(rf_scaler.transform(X_arr)),  zero_division=0)
    xgb_base_f1 = f1_score(y_arr,
xgb_model.predict(xgb_scaler.transform(X_arr)), zero_division=0)
    print(f"  Baseline F1  RF={rf_base_f1:.4f}   XGB={xgb_base_f1:.4f}")

    rf_means,  rf_stds  = [], []
    xgb_means, xgb_stds = [], []

    print(f"\n  {'Noise Level':>12}  {'RF F1 mean':>11} {'RF F1 std':>10}
"
          f"{'XGB F1 mean':>12} {'XGB F1 std':>10}  "
          f"{'RF Δ':>8} {'XGB Δ':>8}")
    print(f"  {'-'*12}  {'-'*11} {'-'*10}  {'-'*12} {'-'*10}  {'-'*8} {'-
'*8}")

    for nl in noise_levels:
        rf_f1s_rep, xgb_f1s_rep = [], []
        for rep in range(N_REPEATS):
            rng   = np.random.default_rng(RANDOM_SEED + rep)
            noise = rng.normal(0, nl * feat_stds, X_arr.shape) if nl > 0
else np.zeros_like(X_arr)
            X_noisy = X_arr + noise

            rf_preds  = rf_model.predict(rf_scaler.transform(X_noisy))
            xgb_preds = xgb_model.predict(xgb_scaler.transform(X_noisy))
            rf_f1s_rep.append(f1_score(y_arr, rf_preds, zero_division=0))
            xgb_f1s_rep.append(f1_score(y_arr, xgb_preds, zero_division=0))

        rf_m,  rf_s  = np.mean(rf_f1s_rep),  np.std(rf_f1s_rep)
        xgb_m, xgb_s = np.mean(xgb_f1s_rep), np.std(xgb_f1s_rep)
        rf_means.append(rf_m);   rf_stds.append(rf_s)
        xgb_means.append(xgb_m); xgb_stds.append(xgb_s)

        delta_rf  = rf_m  - rf_base_f1
```

```
        delta_xgb = xgb_m - xgb_base_f1
        print(f"  {nl:>12.2f}  {rf_m:>11.4f} {rf_s:>10.4f}  "
              f"{xgb_m:>12.4f} {xgb_s:>10.4f}  "
              f"{delta_rf:>+8.4f} {delta_xgb:>+8.4f}")

    # ── Main F1 degradation curve ──
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(noise_levels, rf_means, 'o-',  color=COLORS['primary'],
            linewidth=2.5, markersize=8, label='RF + SMOTE')
    ax.fill_between(noise_levels,
                    [m - s for m, s in zip(rf_means, rf_stds)],
                    [m + s for m, s in zip(rf_means, rf_stds)],
                    color=COLORS['primary'], alpha=0.15)
    ax.plot(noise_levels, xgb_means, 's-', color=COLORS['xgb'],
            linewidth=2.5, markersize=8, label='XGBoost')
    ax.fill_between(noise_levels,
                    [m - s for m, s in zip(xgb_means, xgb_stds)],
                    [m + s for m, s in zip(xgb_means, xgb_stds)],
                    color=COLORS['xgb'], alpha=0.15)
    ax.axhline(y=rf_base_f1,  color=COLORS['primary'],
               linestyle=':', linewidth=1.5, alpha=0.6, label='RF
Baseline')
    ax.axhline(y=xgb_base_f1, color=COLORS['xgb'],
               linestyle=':', linewidth=1.5, alpha=0.6, label='XGB
Baseline')
    ax.set_xlabel('Noise Level (σ × feature std)', fontsize=13,
fontweight='bold')
    ax.set_ylabel('F1 Score (mean ± std, 5 runs)', fontsize=13,
fontweight='bold')
    ax.set_title('F1 Degradation Under Gaussian Feature Noise',
                 fontsize=15, fontweight='bold', pad=12)
    ax.legend(fontsize=11, frameon=True, edgecolor='black')
    ax.grid(linestyle='--', alpha=0.4)
    ax.set_ylim(max(0, min(rf_means + xgb_means) - 0.10), 1.02)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()

    # ── Per-feature noise sensitivity ──
    print("\n  Per-feature F1 degradation (noise_level=0.50, RF, n=5
repeats):")
    pf_results = []
    for fi, fname in enumerate(feature_names):
        rep_f1s = []
        for rep in range(N_REPEATS):
            rng    = np.random.default_rng(RANDOM_SEED + rep + fi * 100)
            X_pf   = X_arr.copy()
            X_pf[:, fi] += rng.normal(0, 0.50 * feat_stds[fi], len(X_arr))
            pred   = rf_model.predict(rf_scaler.transform(X_pf))
            rep_f1s.append(f1_score(y_arr, pred, zero_division=0))
        mean_f1 = np.mean(rep_f1s)
        delta   = mean_f1 - rf_base_f1
```

```
        pf_results.append((fname, mean_f1, delta))
        print(f"    {fname:<40}  F1={mean_f1:.4f}  ΔF1={delta:+.4f}")

    pf_results.sort(key=lambda x: x[2])
    pf_names  = [p[0] for p in pf_results]
    pf_deltas = [p[2] for p in pf_results]

    fig2, ax2 = plt.subplots(figsize=(10, max(4, len(feature_names) *
0.55)))
    bar_colors = [COLORS['danger'] if d < -0.01 else
                  COLORS['warning'] if d < 0 else
                  COLORS['success'] for d in pf_deltas]
    ax2.barh(pf_names, pf_deltas, color=bar_colors,
             edgecolor='black', linewidth=1.2, alpha=0.9)
    ax2.axvline(0, color='black', linewidth=1.5)
    ax2.set_xlabel('ΔF1 (mean F1 with noise – baseline F1)',
                   fontsize=13, fontweight='bold')
    ax2.set_title('Per-Feature F1 Degradation (RF, noise σ=0.50×feature
std)',
                  fontsize=14, fontweight='bold', pad=10)
    ax2.grid(axis='x', linestyle='--', alpha=0.4)
    for bar in ax2.patches:
        w = bar.get_width()
        ax2.annotate(f'{w:+.4f}',
                     xy=(w, bar.get_y() + bar.get_height() / 2),
                     xytext=(3 if w >= 0 else -3, 0),
                     textcoords='offset points',
                     va='center', ha='left' if w >= 0 else 'right',
                     fontsize=10, fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path.replace('.png', '_per_feature.png'),
                    dpi=300, bbox_inches='tight')
    plt.show()

    return {"noise_levels": noise_levels,
            "rf_means":   rf_means,   "rf_stds":   rf_stds,
            "xgb_means":  xgb_means,  "xgb_stds":  xgb_stds,
            "per_feature": pf_results}

# ============================================================
# ADVERSARIAL EVASION TEST
# ============================================================
def adversarial_evasion_test(rf_model, rf_scaler,
                             xgb_model, xgb_scaler,
                             X_test: pd.DataFrame,
                             y_test: pd.Series,
                             feature_names: List[str],
                             n_attack_samples: int = 300,
                             epsilons: Tuple = (0.0, 0.05, 0.10, 0.20,
0.40, 0.80),
                             save_path: str = None):
    """
```

```
    Simple perturbation-based adversarial evasion test.
    Measures fraction of detected attacks that get misclassified under
increasing noise.
    """
    print_section("ADVERSARIAL EVASION TEST")

    # Select attack samples
    attack_mask = y_test.values == 1
    X_attack = X_test[attack_mask].copy().reset_index(drop=True)
    if len(X_attack) == 0:
        print("  No attack samples found. Skipping.")
        return
    X_attack = X_attack.sample(min(n_attack_samples, len(X_attack)),
                               random_state=RANDOM_SEED)
    print(f"  Attack samples selected for evasion test: {len(X_attack)}")

    feature_stds = X_test.std(axis=0).values + 1e-9

    rf_evasion  = []
    xgb_evasion = []

    for eps in epsilons:
        if eps == 0.0:
            X_adv = X_attack.values.copy()
        else:
            rng   = np.random.default_rng(RANDOM_SEED)
            noise = rng.normal(0, eps * feature_stds, X_attack.shape)
            X_adv = X_attack.values + noise

        # RF evasion
        X_adv_sc_rf  = rf_scaler.transform(X_adv)
        pred_rf  = rf_model.predict(X_adv_sc_rf)
        rf_evasion.append(float((pred_rf == 0).mean()))

        # XGB evasion
        X_adv_sc_xgb = xgb_scaler.transform(X_adv)
        pred_xgb = xgb_model.predict(X_adv_sc_xgb)
        xgb_evasion.append(float((pred_xgb == 0).mean()))

    # Report
    print(f"\n  {'Epsilon':>8}  {'RF Evasion Rate':>16}  {'XGB Evasion
Rate':>17}")
    print(f"  {'-'*8}  {'-'*16}  {'-'*17}")
    for eps, rf_ev, xgb_ev in zip(epsilons, rf_evasion, xgb_evasion):
        print(f"  {eps:>8.2f}  {rf_ev:>16.4%}  {xgb_ev:>17.4%}")

    # Plot
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(epsilons, [v * 100 for v in rf_evasion],
            'o-', color=COLORS['primary'],  linewidth=2.5,
            markersize=8, label='RF + SMOTE')
    ax.plot(epsilons, [v * 100 for v in xgb_evasion],
            's-', color=COLORS['xgb'],      linewidth=2.5,
```

```
            markersize=8, label='XGBoost')
    ax.set_xlabel('Perturbation Budget ε (fraction of feature std)',
                  fontsize=13, fontweight='bold')
    ax.set_ylabel('Evasion Rate (%)', fontsize=13, fontweight='bold')
    ax.set_title('Adversarial Evasion Rate vs Perturbation Budget',
                 fontsize=15, fontweight='bold', pad=12)
    ax.legend(fontsize=12, frameon=True, edgecolor='black')
    ax.grid(linestyle='--', alpha=0.4)
    ax.set_ylim(0, 105)
    for x_val, y_val in zip(epsilons, [v * 100 for v in rf_evasion]):
        ax.annotate(f'{y_val:.1f}%', xy=(x_val, y_val),
                    xytext=(0, 8), textcoords='offset points',
                    ha='center', fontsize=10, color=COLORS['primary'],
fontweight='bold')
    for x_val, y_val in zip(epsilons, [v * 100 for v in xgb_evasion]):
        ax.annotate(f'{y_val:.1f}%', xy=(x_val, y_val),
                    xytext=(0, -14), textcoords='offset points',
                    ha='center', fontsize=10, color=COLORS['xgb'],
fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()

    # Per-feature sensitivity
    print("\n  Per-feature evasion sensitivity (epsilon=0.20, RF):")
    per_feat_evasion = []
    for fi, fname in enumerate(feature_names):
        rng   = np.random.default_rng(RANDOM_SEED + fi)
        X_adv_f = X_attack.values.copy().astype(float)
        X_adv_f[:, fi] += rng.normal(0, 0.20 * feature_stds[fi],
len(X_attack))
        pred_f = rf_model.predict(rf_scaler.transform(X_adv_f))
        ev_f   = float((pred_f == 0).mean())
        per_feat_evasion.append((fname, ev_f))
        print(f"    {fname:<40}  evasion={ev_f:.4%}")

    per_feat_evasion.sort(key=lambda x: x[1], reverse=True)
    feat_names_sorted = [p[0] for p in per_feat_evasion]
    feat_ev_sorted    = [p[1] * 100 for p in per_feat_evasion]

    fig2, ax2 = plt.subplots(figsize=(10, max(4, len(feature_names) *
0.55)))
    ax2.barh(feat_names_sorted, feat_ev_sorted,
             color=COLORS['danger'], edgecolor='black', linewidth=1.2,
alpha=0.9)
    ax2.set_xlabel('Evasion Rate (%)', fontsize=13, fontweight='bold')
    ax2.set_title('Per-Feature Evasion Sensitivity (RF, ε=0.20)',
                  fontsize=14, fontweight='bold', pad=10)
    ax2.invert_yaxis()
    ax2.grid(axis='x', linestyle='--', alpha=0.4)
    for bar in ax2.patches:
        w = bar.get_width()
```

```
        ax2.annotate(f'{w:.1f}%',
                     xy=(w, bar.get_y() + bar.get_height() / 2),
                     xytext=(3, 0), textcoords='offset points',
                     va='center', fontsize=10, fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path.replace('.png', '_per_feature.png'),
                    dpi=300, bbox_inches='tight')
    plt.show()

    return {"epsilons": epsilons, "rf_evasion": rf_evasion,
            "xgb_evasion": xgb_evasion, "per_feature": per_feat_evasion}

# ============================================================
# SHAP EXPLANATIONS FOR HITL ESCALATIONS
# ============================================================
def explain_hitl_with_shap(model, scaler, hitl_logs: List[Dict],
                           feature_names: List[str],
                           model_name: str = "RF",
                           max_display: int = 10,
                           max_samples: int = 200,
                           save_path_summary: str = None,
                           save_path_bar: str = None):
    """
    Compute and plot SHAP explanations for HITL-escalated samples.
    Uses TreeExplainer (fast, exact for tree-based models).
    """
    print_section(f"SHAP EXPLANATIONS FOR HITL ESCALATIONS — {model_name}")
    if not hitl_logs:
        print("  No HITL samples to explain.")
        return

    # Build feature matrix from stored logs
    rows = []
    for log in hitl_logs[:max_samples]:
        row_data = [safe_float(log.get(feat, 0.0)) for feat in
feature_names]
        rows.append(row_data)

    if not rows:
        print("  Could not extract features from HITL logs.")
        return

    X_hitl = pd.DataFrame(rows, columns=feature_names)
    X_hitl_scaled = scaler.transform(X_hitl)

    try:
        with LATENCY.measure(f"SHAP TreeExplainer ({model_name})"):
            explainer   = shap.TreeExplainer(model)
            shap_values = explainer.shap_values(X_hitl_scaled)

        # For binary classifiers, shap_values may be a list
        if isinstance(shap_values, list):
```

```
            sv = shap_values[1]  # attack class
        else:
            sv = shap_values

        print(f"  HITL samples explained : {X_hitl_scaled.shape[0]}")
        print(f"  Feature count          : {X_hitl_scaled.shape[1]}")

        # --- Summary beeswarm plot ---
        try:
            fig1, ax1 = plt.subplots(figsize=(10, 6))
            shap.summary_plot(sv, X_hitl_scaled,
                              feature_names=feature_names,
                              max_display=max_display,
                              show=False)
            plt.title(f"SHAP Summary — HITL Escalations ({model_name})",
                      fontsize=15, fontweight='bold', pad=12)
            plt.tight_layout()
            if save_path_summary:
                plt.savefig(save_path_summary, dpi=300,
bbox_inches='tight')
            plt.show()
        except Exception as e:
            print(f"  SHAP summary plot failed: {e}")

        # --- Mean |SHAP| bar chart ---
        mean_abs_shap = np.abs(sv).mean(axis=0)
        indices = np.argsort(mean_abs_shap)[::-1]
        fig2, ax2 = plt.subplots(figsize=(10, 5))
        ax2.barh(range(len(feature_names)),
                 mean_abs_shap[indices],
                 color=COLORS['primary'], edgecolor='black',
                 linewidth=1.3, alpha=0.9)
        ax2.set_yticks(range(len(feature_names)))
        ax2.set_yticklabels([feature_names[i] for i in indices],
fontsize=12)
        ax2.set_xlabel("Mean |SHAP value|", fontsize=13, fontweight='bold')
        ax2.set_title(f"Feature Impact on HITL Escalation ({model_name})",
                      fontsize=15, fontweight='bold', pad=12)
        ax2.invert_yaxis()
        ax2.grid(axis='x', linestyle='--', alpha=0.35)
        for j, bar in enumerate(ax2.patches):
            w = bar.get_width()
            ax2.annotate(f'{w:.4f}',
                         xy=(w, bar.get_y() + bar.get_height() / 2),
                         xytext=(3, 0), textcoords='offset points',
                         va='center', fontsize=10, fontweight='bold')
        plt.tight_layout()
        if save_path_bar:
            plt.savefig(save_path_bar, dpi=300, bbox_inches='tight')
        plt.show()

        # Top-3 drivers
        print("\n  Top-3 SHAP drivers for HITL escalations:")
```

```
        for rank, idx in enumerate(indices[:3], 1):
            print(f"    {rank}. {feature_names[idx]:<35} mean|SHAP|=
{mean_abs_shap[idx]:.5f}")

        return sv

    except Exception as e:
        print(f"  SHAP computation failed: {e}")
        return None

# ============================================================
# PLOTTING FUNCTIONS — All publication-quality figures
# ============================================================
def plot_model_performance_comparison(metrics_rf, metrics_if, metrics_xgb,
                                      save_path=None):
    """Publication-quality model performance comparison — 3 models."""
    fig, ax = plt.subplots(figsize=(13, 7))
    metrics_list = ['Accuracy', 'Precision', 'Recall', 'F1-score']
    vals_rf  = [metrics_rf['Accuracy'],  metrics_rf['Precision'],
                metrics_rf['Recall'],  metrics_rf['F1']]
    vals_if  = [metrics_if['Accuracy'],  metrics_if['Precision'],
                metrics_if['Recall'],  metrics_if['F1']]
    vals_xgb = [metrics_xgb['Accuracy'], metrics_xgb['Precision'],
                metrics_xgb['Recall'], metrics_xgb['F1']]
    x = np.arange(len(metrics_list))
    width = 0.26

    for vals, label, color, offset in [
        (vals_rf,  'RF + SMOTE',      COLORS['primary'],   -width),
        (vals_if,  'Isolation Forest', COLORS['secondary'],  0),
        (vals_xgb, 'XGBoost',          COLORS['xgb'],       +width),
    ]:
        bars = ax.bar(x + offset, vals, width, label=label,
                      color=color, edgecolor='black', linewidth=1.4,
alpha=0.9)
        for bar in bars:
            height = bar.get_height()
            ax.text(bar.get_x() + bar.get_width() / 2.0, height,
f'{height:.3f}',
                    ha='center', va='bottom', fontsize=10,
fontweight='bold')

    ax.set_ylabel('Score', fontsize=14, fontweight='bold')
    ax.set_xlabel('Evaluation Metric', fontsize=14, fontweight='bold')
    ax.set_title('Model Performance Comparison (3 Models)',
                 fontsize=16, fontweight='bold', pad=20)
    ax.set_xticks(x); ax.set_xticklabels(metrics_list, fontsize=12)
    ax.set_ylim(0, 1.12)
    ax.grid(axis='y', linestyle='--', alpha=0.3, linewidth=1)
    ax.legend(fontsize=12, frameon=True, edgecolor='black', loc='lower
right')
    plt.tight_layout()
    if save_path:
```

```
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_confusion_matrix_combined(cm_rf, cm_if, cm_xgb=None,
save_path=None):
    """Confusion matrices for all models."""
    n_plots = 3 if cm_xgb is not None else 2
    fig, axes = plt.subplots(1, n_plots, figsize=(7 * n_plots, 6))
    if n_plots == 2:
        axes = [axes[0], axes[1]]

    cms    = [cm_rf, cm_if] + ([cm_xgb] if cm_xgb is not None else [])
    titles = ['RF + SMOTE', 'Isolation Forest'] + (['XGBoost'] if cm_xgb is
not None else [])

    for ax, cm, title in zip(axes, cms, titles):
        im = ax.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues,
alpha=0.85)
        ax.figure.colorbar(im, ax=ax)
        ax.set(xticks=np.arange(2), yticks=np.arange(2),
               xticklabels=['Benign', 'Attack'],
               yticklabels=['Benign', 'Attack'],
               ylabel='True Label', xlabel='Predicted Label')
        ax.set_title(title, fontsize=14, fontweight='bold', pad=12)
        thresh = cm.max() / 2.
        for i in range(2):
            for j in range(2):
                ax.text(j, i, format(cm[i, j], 'd'),
                        ha="center", va="center",
                        color="white" if cm[i, j] > thresh else "black",
                        fontsize=14, fontweight='bold')

    plt.suptitle('Confusion Matrix Comparison',
                 fontsize=18, fontweight='bold', y=1.02)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_roc_curves(y_test, rf_model, rf_scaler, if_model,
                    xgb_model, xgb_scaler, X_test, save_path=None):
    """ROC curves for RF, IF, XGBoost."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # RF
    y_rf  = rf_model.predict_proba(rf_scaler.transform(X_test))[:, 1]
    fpr_rf, tpr_rf, _ = roc_curve(y_test, y_rf)
    auc_rf = auc(fpr_rf, tpr_rf)
    ax.plot(fpr_rf, tpr_rf, color=COLORS['primary'], linewidth=3,
            label=f'RF + SMOTE (AUC={auc_rf:.4f})')
```

```
    # IF
    y_if  = -if_model.decision_function(X_test)
    y_if  = (y_if - y_if.min()) / (y_if.max() - y_if.min() + 1e-9)
    fpr_if, tpr_if, _ = roc_curve(y_test, y_if)
    auc_if = auc(fpr_if, tpr_if)
    ax.plot(fpr_if, tpr_if, color=COLORS['secondary'], linewidth=3,
            label=f'Isolation Forest (AUC={auc_if:.4f})')

    # XGB
    y_xgb = xgb_model.predict_proba(xgb_scaler.transform(X_test))[:, 1]
    fpr_xgb, tpr_xgb, _ = roc_curve(y_test, y_xgb)
    auc_xgb = auc(fpr_xgb, tpr_xgb)
    ax.plot(fpr_xgb, tpr_xgb, color=COLORS['xgb'], linewidth=3,
            label=f'XGBoost (AUC={auc_xgb:.4f})')

    ax.plot([0, 1], [0, 1], 'k--', linewidth=2, alpha=0.5, label='Random')
    ax.set_xlim([0.0, 1.0]); ax.set_ylim([0.0, 1.05])
    ax.set_xlabel('False Positive Rate', fontsize=14, fontweight='bold')
    ax.set_ylabel('True Positive Rate',  fontsize=14, fontweight='bold')
    ax.set_title('ROC Curve Comparison', fontsize=16, fontweight='bold',
pad=15)
    ax.legend(loc='lower right', fontsize=12, frameon=True,
edgecolor='black')
    ax.grid(alpha=0.3, linestyle='--')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_feature_importance(rf_model, xgb_model, feature_names,
save_path=None):
    """Side-by-side feature importance for RF and XGBoost."""
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))
    for ax, model, color, title in [
        (axes[0], rf_model,  COLORS['primary'], 'Random Forest Feature
Importance'),
        (axes[1], xgb_model, COLORS['xgb'],     'XGBoost Feature
Importance'),
    ]:
        imps = model.feature_importances_
        idx  = np.argsort(imps)[::-1]
        ax.barh(range(len(imps)), imps[idx], color=color,
                edgecolor='black', linewidth=1.3, alpha=0.9)
        ax.set_yticks(range(len(imps)))
        ax.set_yticklabels([feature_names[i] for i in idx], fontsize=12)
        ax.set_xlabel('Importance Score', fontsize=13, fontweight='bold')
        ax.set_title(title, fontsize=14, fontweight='bold', pad=12)
        ax.invert_yaxis()
        ax.grid(axis='x', linestyle='--', alpha=0.35)
        for bar in ax.patches:
            w = bar.get_width()
            ax.annotate(f'{w:.4f}',
```

```
                        xy=(w, bar.get_y() + bar.get_height() / 2),
                        xytext=(4, 0), textcoords='offset points',
                        va='center', fontsize=10, fontweight='bold')

    plt.suptitle('Feature Importance Comparison',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_agentic_decisions_comparison(tracker_rf, tracker_if,
                                      tracker_xgb=None, save_path=None):
    """AUTO vs HITL decision comparison."""
    trackers = [tracker_rf, tracker_if]
    if tracker_xgb:
        trackers.append(tracker_xgb)

    names      = [t.name for t in trackers]
    auto_cnts  = [t.auto_actions for t in trackers]
    hitl_cnts  = [t.hitl_actions for t in trackers]
    auto_rates = [(t.auto_actions / (t.auto_actions + t.hitl_actions)
                   if (t.auto_actions + t.hitl_actions) > 0 else 0) for t
in trackers]

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
    x = np.arange(len(names)); width = 0.35

    b1 = axes[0].bar(x - width/2, auto_cnts, width, label='AUTO',
                     color=COLORS['success'], edgecolor='black',
linewidth=1.4, alpha=0.9)
    b2 = axes[0].bar(x + width/2, hitl_cnts, width, label='HITL',
                     color=COLORS['danger'],  edgecolor='black',
linewidth=1.4, alpha=0.9)
    axes[0].set_xticks(x)
    axes[0].set_xticklabels(names, fontsize=11, rotation=15)
    axes[0].set_title('Automated vs Human-in-the-Loop Decisions',
                      fontsize=14, fontweight='bold', pad=12)
    axes[0].set_ylabel('Count', fontsize=13, fontweight='bold')
    axes[0].legend(fontsize=11); axes[0].grid(axis='y', linestyle='--',
alpha=0.35)
    for bars in [b1, b2]:
        for bar in bars:
            height = bar.get_height()
            axes[0].annotate(f'{int(height)}',
                             xy=(bar.get_x() + bar.get_width() / 2,
height),
                             xytext=(0, 3), textcoords='offset points',
                             ha='center', va='bottom', fontsize=11,
fontweight='bold')

    bar_colors = [COLORS['success'] if r > 0.1 else COLORS['neutral'] for r
```

```
in auto_rates]
    b3 = axes[1].bar(names, auto_rates, color=bar_colors,
                     edgecolor='black', linewidth=1.4, alpha=0.9)
    axes[1].set_title('Automation Rate by Pipeline',
                      fontsize=14, fontweight='bold', pad=12)
    axes[1].set_ylabel('AUTO Rate', fontsize=13, fontweight='bold')
    axes[1].set_ylim(0, 1.05)
    axes[1].axhline(0.2, color='gray', linestyle='--', linewidth=1.5,
                    alpha=0.5, label='20% Target')
    axes[1].legend(fontsize=11); axes[1].grid(axis='y', linestyle='--',
alpha=0.35)
    for bar in b3:
        height = bar.get_height()
        axes[1].annotate(f'{height:.2%}',
                         xy=(bar.get_x() + bar.get_width() / 2, height),
                         xytext=(0, 3), textcoords='offset points',
                         ha='center', va='bottom', fontsize=11,
fontweight='bold')

    plt.suptitle('Agentic Pipeline Decision Analysis',
                 fontsize=18, fontweight='bold', y=1.02)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_confidence_novelty_scatter(tracker_rf, tracker_if,
                                    tracker_xgb=None, save_path=None):
    """Confidence vs Novelty scatter for all pipelines."""
    fig, ax = plt.subplots(figsize=(10, 8))
    plot_list = [
        (tracker_rf,  COLORS['primary'],   'o', f'RF + SMOTE (n=
{len(tracker_rf.confidence_scores)})'),
        (tracker_if,  COLORS['secondary'], 's', f'Isolation Forest (n=
{len(tracker_if.confidence_scores)})'),
    ]
    if tracker_xgb:
        plot_list.append(
            (tracker_xgb, COLORS['xgb'], '^', f'XGBoost (n=
{len(tracker_xgb.confidence_scores)})')
        )

    for t, color, marker, label in plot_list:
        ax.scatter(t.confidence_scores, t.novelty_scores,
                   c=color, label=label, marker=marker,
                   alpha=0.6, s=80, edgecolors='black', linewidth=0.5)

    ax.axvline(x=0.80, color=COLORS['danger'],  linestyle='--',
linewidth=2.5,
               label='Conf Threshold (0.80)', alpha=0.8)
    ax.axhline(y=0.95, color=COLORS['warning'], linestyle='--',
linewidth=2.5,
```

```
                label='Novelty Threshold (0.95)', alpha=0.8)
    ax.set_xlabel('Confidence Score', fontsize=14, fontweight='bold')
    ax.set_ylabel('Novelty Score',    fontsize=14, fontweight='bold')
    ax.set_title('Decision Boundary: Confidence vs Novelty',
                 fontsize=16, fontweight='bold', pad=15)
    ax.legend(fontsize=11, frameon=True, edgecolor='black')
    ax.grid(alpha=0.3, linestyle='--')
    ax.set_xlim(0, 1.05); ax.set_ylim(0, 1.05)
    ax.annotate('AUTO Region', xy=(0.90, 0.5), fontsize=12,
fontweight='bold',
                color=COLORS['success'], alpha=0.8)
    ax.annotate('HITL Region', xy=(0.5, 0.97), fontsize=12,
fontweight='bold',
                color=COLORS['danger'], alpha=0.8)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_severity_distribution(tracker_rf, tracker_if,
                               tracker_xgb=None, save_path=None):
    """Severity distribution by pipeline."""
    severity_order = ['low', 'medium', 'high', 'critical']
    trackers = [tracker_rf, tracker_if]
    if tracker_xgb:
        trackers.append(tracker_xgb)

    n = len(trackers)
    x = np.arange(len(severity_order))
    width = 0.25 if n == 3 else 0.35
    offsets = np.linspace(-(n-1)*width/2, (n-1)*width/2, n)

    fig, ax = plt.subplots(figsize=(12, 7))
    colors = [COLORS['primary'], COLORS['secondary'], COLORS['xgb']]

    for t, color, offset in zip(trackers, colors, offsets):
        vals = [t.severity_count.get(s, 0) for s in severity_order]
        bars = ax.bar(x + offset, vals, width, label=t.name,
                      color=color, edgecolor='black', linewidth=1.3,
alpha=0.9)
        for bar in bars:
            height = bar.get_height()
            if height > 0:
                ax.annotate(f'{int(height)}',
                            xy=(bar.get_x() + bar.get_width() / 2, height),
                            xytext=(0, 3), textcoords='offset points',
                            ha='center', va='bottom', fontsize=10,
fontweight='bold')

    ax.set_xticks(x)
    ax.set_xticklabels([s.capitalize() for s in severity_order],
fontsize=12)
```

```
    ax.set_ylabel('Incident Count', fontsize=14, fontweight='bold')
    ax.set_xlabel('Severity Level',  fontsize=14, fontweight='bold')
    ax.set_title('Incident Severity Distribution by Pipeline',
                 fontsize=16, fontweight='bold', pad=15)
    ax.legend(fontsize=12, frameon=True, edgecolor='black')
    ax.grid(axis='y', linestyle='--', alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_incident_metrics_comparison(tracker_rf, tracker_if,
                                     tracker_xgb=None, save_path=None):
    """Incident precision by pipeline."""
    trackers = [tracker_rf, tracker_if]
    if tracker_xgb:
        trackers.append(tracker_xgb)

    names  = [t.name for t in trackers]
    precs  = [(t.incident_tp / (t.incident_tp + t.incident_fp)
               if (t.incident_tp + t.incident_fp) > 0 else 0) for t in
trackers]
    colors = [COLORS['primary'], COLORS['secondary'], COLORS['xgb']]
[:len(trackers)]

    fig, ax = plt.subplots(figsize=(10, 7))
    bars = ax.bar(names, precs, color=colors, edgecolor='black',
                  linewidth=1.4, alpha=0.9)
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.2%}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), textcoords='offset points',
                    ha='center', va='bottom', fontsize=12,
fontweight='bold')

    ax.set_ylabel('Incident Precision', fontsize=14, fontweight='bold')
    ax.set_xlabel('Pipeline',           fontsize=14, fontweight='bold')
    ax.set_title('Incident-Level Precision Comparison',
                 fontsize=16, fontweight='bold', pad=15)
    ax.set_ylim(0, 1.05)
    ax.axhline(0.5, color='gray', linestyle='--', linewidth=1.5,
               alpha=0.5, label='50% Baseline')
    ax.legend(fontsize=12); ax.grid(axis='y', linestyle='--', alpha=0.3)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def plot_cv_results(cv_scores_rf, cv_scores_if, cv_scores_xgb=None,
save_path=None):
```

```
    """Cross-validation results for all models."""
    models = ['RF + SMOTE', 'Isolation Forest']
    means  = [np.mean(cv_scores_rf), np.mean(cv_scores_if)]
    stds   = [np.std(cv_scores_rf),  np.std(cv_scores_if)]
    colors = [COLORS['primary'], COLORS['secondary']]

    if cv_scores_xgb is not None and len(cv_scores_xgb) > 0:
        models.append('XGBoost')
        means.append(np.mean(cv_scores_xgb))
        stds.append(np.std(cv_scores_xgb))
        colors.append(COLORS['xgb'])

    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.bar(np.arange(len(models)), means, yerr=stds, capsize=5,
                  color=colors, edgecolor='black', linewidth=1.4,
alpha=0.9)
    ax.set_ylabel('F1 Score', fontsize=14, fontweight='bold')
    ax.set_xlabel('Model',    fontsize=14, fontweight='bold')
    ax.set_title('3-Fold Cross-Validation Performance (F1 Score)',
                 fontsize=16, fontweight='bold', pad=15)
    ax.set_xticks(np.arange(len(models)))
    ax.set_xticklabels(models, fontsize=12)
    ax.set_ylim(0, 1.05)
    ax.grid(axis='y', linestyle='--', alpha=0.3)
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.3f}',
                    xy=(bar.get_x() + bar.get_width() / 2, height),
                    xytext=(0, 3), textcoords='offset points',
                    ha='center', va='bottom', fontsize=11,
fontweight='bold')
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
    return fig

def print_dataset_overview(df, feature_cols):
    """Print dataset summary statistics."""
    print_section("DATASET OVERVIEW")
    print(f"Rows after preprocessing : {len(df)}")
    print("\nLabel distribution:")
    print(df[LABEL_COLUMN].value_counts().to_string())
    print("\nSample feature statistics:")
    available_features = [f for f in feature_cols if f in df.columns]
    if available_features:
        stats = df[available_features].describe().T[["mean", "std", "min",
"max"]]
        print(stats.to_string())

# ============================================================
# MAIN EXECUTION
# ============================================================
```

```
def main():
    """Complete end-to-end anomaly detection pipeline."""

    # ── 1. LOAD & PREPROCESS ──────────────────────────────────
    print_section("LOADING AND PREPROCESSING DATA (500k samples)")
    raw_df = load_data(DATASET_PATHS)

    # Preprocess with only available features (no zero-filling)
    df, available_features = preprocess(raw_df.copy(), sample_size=500000,
                                        feature_cols=FEATURE_COLUMNS)
    print_dataset_overview(df, available_features)

    # Also load full feature dataset for subset experiments
    df_full, all_avail_features = preprocess(raw_df.copy(),
sample_size=500000,

feature_cols=ALL_POSSIBLE_FEATURES)

    # Split data
    y_all   = df[LABEL_COLUMN].map(label_to_binary)
    train_df, test_df = train_test_split(
        df, test_size=0.30, stratify=y_all, random_state=RANDOM_SEED)

    X_train = train_df[available_features]
    y_train = train_df[LABEL_COLUMN].map(label_to_binary)
    X_test  = test_df[available_features]
    y_test  = test_df[LABEL_COLUMN].map(label_to_binary)

    # ── 2. TRAIN MODELS ───────────────────────────────────────
    print_section("TRAINING MODELS")
    print("Building IsolationForest...")
    if_model = build_iforest(X_train)
    print("Building RF + SMOTE...")
    rf_model, rf_scaler = build_rf_smote(X_train, y_train)
    print("Building XGBoost + SMOTE...")
    xgb_model, xgb_scaler = build_xgboost(X_train, y_train)

    # ── 3. BATCH INFERENCE LATENCY MEASUREMENT ────────────────
    print_section("BATCH INFERENCE LATENCY MEASUREMENT")
    BATCH_SIZES = [1, 10, 100, 1000]
    for bs in BATCH_SIZES:
        sample = X_test.iloc[:bs]
        with LATENCY.measure(f"RF Batch Inference (n={bs})"):
            _ = rf_model.predict(rf_scaler.transform(sample))
        with LATENCY.measure(f"XGB Batch Inference (n={bs})"):
            _ = xgb_model.predict(xgb_scaler.transform(sample))
        with LATENCY.measure(f"IF Batch Inference (n={bs})"):
            _ = if_model.predict(sample)
    LATENCY.report()

    # ── 4. CROSS-VALIDATION ───────────────────────────────────
    print_section("CROSS-VALIDATION (3-FOLD)")
    rf_cv_estimator = ImbPipeline([
```

```
        ('scaler', StandardScaler()),
        ('smote',  SMOTE(random_state=RANDOM_SEED)),
        ('rf',     RandomForestClassifier(
            n_estimators=100, max_depth=8,
            min_samples_split=10, min_samples_leaf=5,
            random_state=RANDOM_SEED, n_jobs=-1))
    ])

    if_cv_estimator = Pipeline([
        ('scaler',  StandardScaler()),
        ('iforest', IsolationForest(
            n_estimators=200, contamination=0.15,
            random_state=RANDOM_SEED))
    ])

    xgb_cv_estimator = ImbPipeline([
        ('scaler', StandardScaler()),
        ('smote',  SMOTE(random_state=RANDOM_SEED)),
        ('xgb',    xgb.XGBClassifier(
            n_estimators=100, max_depth=6,
            learning_rate=0.1, random_state=RANDOM_SEED,
            eval_metric='logloss', verbosity=0, n_jobs=-1))
    ])

    cv_scores_rf  = perform_cross_validation(X_train, y_train,
                                             "Random Forest + SMOTE",
rf_cv_estimator)
    cv_scores_if  = perform_cross_validation(X_train, y_train,
                                             "Isolation Forest",
if_cv_estimator)
    cv_scores_xgb = perform_cross_validation(X_train, y_train,
                                             "XGBoost",
xgb_cv_estimator)

    # ── 5. MODEL EVALUATION ──────────────────────────────────
    print_section("MODEL EVALUATION ON HOLD-OUT TEST SET")
    with LATENCY.measure("RF Full Test Inference"):
        y_pred_rf  = rf_model.predict(rf_scaler.transform(X_test))
    metrics_rf = print_metrics("Random Forest + SMOTE", y_test, y_pred_rf)

    with LATENCY.measure("IF Full Test Inference"):
        if_pred_raw = if_model.predict(X_test)
    y_pred_if  = (if_pred_raw == -1).astype(int)
    metrics_if = print_metrics("Isolation Forest", y_test, y_pred_if)

    with LATENCY.measure("XGB Full Test Inference"):
        y_pred_xgb = xgb_model.predict(xgb_scaler.transform(X_test))
    metrics_xgb = print_metrics("XGBoost", y_test, y_pred_xgb)

    # ── 6. FEATURE SUBSET EXPERIMENT ─────────────────────────
    feature_subset_results = run_feature_subset_experiment(
        df_full, all_avail_features,
save_path="/content/feature_subset_experiment.png")
```

```
# ── 7. GAUSSIAN NOISE PERTURBATION TEST ──────────────────
noise_results = gaussian_noise_perturbation_test(
    rf_model, rf_scaler, xgb_model, xgb_scaler,
    X_test, y_test, available_features,
    save_path="/content/noise_perturbation.png")

# ── 8. ADVERSARIAL EVASION TEST ──────────────────────────
evasion_results = adversarial_evasion_test(
    rf_model, rf_scaler, xgb_model, xgb_scaler,
    X_test, y_test, available_features,
    save_path="/content/adversarial_evasion.png")

# ── 9. PUBLICATION-QUALITY FIGURES ───────────────────────
print_section("GENERATING PUBLICATION-QUALITY FIGURES")
plot_model_performance_comparison(
    metrics_rf, metrics_if, metrics_xgb,
    save_path="/content/model_performance.png")
plot_confusion_matrix_combined(
    metrics_rf["Confusion Matrix"],
    metrics_if["Confusion Matrix"],
    metrics_xgb["Confusion Matrix"],
    save_path="/content/confusion_matrices.png")
plot_roc_curves(
    y_test, rf_model, rf_scaler, if_model,
    xgb_model, xgb_scaler, X_test,
    save_path="/content/roc_curves.png")
plot_feature_importance(
    rf_model, xgb_model, available_features,
    save_path="/content/feature_importance.png")
plot_cv_results(cv_scores_rf, cv_scores_if, cv_scores_xgb,
                save_path="/content/cv_results.png")
LATENCY.plot(save_path="/content/latency_profile.png")

# ── 10. AGENTIC PIPELINE RUNS ────────────────────────────
test_logs = df_to_logs(test_df, available_features)

print_section("AGENTIC PIPELINE RUN — RANDOM FOREST + SMOTE")
orch_rf = Orchestrator(rf_model, "rf", rf_scaler, name="RF + SMOTE",
                       feature_names=available_features)
run_rf  = orch_rf.run(test_logs)
tracker_rf = run_rf["tracker"]
tracker_rf.report()
tracker_rf.plot(prefix="RF + SMOTE")

print_section("AGENTIC PIPELINE RUN — ISOLATION FOREST")
orch_if = Orchestrator(if_model, "iforest", name="IsolationForest",
                       feature_names=available_features)
run_if  = orch_if.run(test_logs)
tracker_if = run_if["tracker"]
tracker_if.report()
tracker_if.plot(prefix="IsolationForest")
```

```
    print_section("AGENTIC PIPELINE RUN — XGBOOST")
    orch_xgb = Orchestrator(xgb_model, "xgb", xgb_scaler, name="XGBoost",
                            feature_names=available_features)
    run_xgb  = orch_xgb.run(test_logs)
    tracker_xgb = run_xgb["tracker"]
    tracker_xgb.report()
    tracker_xgb.plot(prefix="XGBoost")

    # ── 11. SHAP EXPLANATIONS FOR HITL ──────────────────────
    print_section("SHAP EXPLANATIONS FOR HITL ESCALATIONS")
    explain_hitl_with_shap(
        rf_model, rf_scaler,
        tracker_rf.hitl_sample_rows,
        available_features, model_name="RF + SMOTE",
        save_path_summary="/content/shap_summary_rf.png",
        save_path_bar="/content/shap_bar_rf.png")
    explain_hitl_with_shap(
        xgb_model, xgb_scaler,
        tracker_xgb.hitl_sample_rows,
        available_features, model_name="XGBoost",
        save_path_summary="/content/shap_summary_xgb.png",
        save_path_bar="/content/shap_bar_xgb.png")

    # ── 12. AGENTIC PIPELINE FIGURES ────────────────────────
    print_section("GENERATING AGENTIC PIPELINE FIGURES")
    plot_agentic_decisions_comparison(
        tracker_rf, tracker_if, tracker_xgb,
        save_path="/content/agentic_decisions.png")
    plot_confidence_novelty_scatter(
        tracker_rf, tracker_if, tracker_xgb,
        save_path="/content/decision_boundary.png")
    plot_severity_distribution(
        tracker_rf, tracker_if, tracker_xgb,
        save_path="/content/severity_distribution.png")
    plot_incident_metrics_comparison(
        tracker_rf, tracker_if, tracker_xgb,
        save_path="/content/incident_precision.png")

    # ── 13. FINAL SUMMARY TABLES ────────────────────────────
    print_section("FINAL SUMMARY TABLES")
    model_summary = pd.DataFrame([
        {"Model": "RF + SMOTE",       "Accuracy": metrics_rf["Accuracy"],
         "Precision": metrics_rf["Precision"], "Recall":
metrics_rf["Recall"],
         "F1-score": metrics_rf["F1"]},
        {"Model": "Isolation Forest", "Accuracy": metrics_if["Accuracy"],
         "Precision": metrics_if["Precision"], "Recall":
metrics_if["Recall"],
         "F1-score": metrics_if["F1"]},
        {"Model": "XGBoost",          "Accuracy": metrics_xgb["Accuracy"],
         "Precision": metrics_xgb["Precision"], "Recall":
metrics_xgb["Recall"],
         "F1-score": metrics_xgb["F1"]},
```

```
    ])
    print("Model Performance Summary:")
    print(model_summary.to_string(index=False))

    agentic_summary = pd.DataFrame([
        tracker_rf.summary_dict(),
        tracker_if.summary_dict(),
        tracker_xgb.summary_dict(),
    ])
    print("\nAgentic Pipeline Summary:")
    print(agentic_summary.to_string(index=False))

    latency_df = pd.DataFrame(LATENCY.summary_rows())
    print("\nLatency Summary:")
    print(latency_df.to_string(index=False))

    if feature_subset_results:
        print("\nFeature Subset Experiment Summary:")
        fs_rows = []
        for sname, r in feature_subset_results.items():
            fs_rows.append({
                "Subset":  sname, "N_feat": r["n_features"],
                "RF_F1":   round(r["RF_F1"],  4), "RF_Acc":
round(r["RF_Acc"],  4),
                "XGB_F1":  round(r["XGB_F1"], 4), "XGB_Acc":
round(r["XGB_Acc"], 4),
                "RF_TrainT_s":  round(r["RF_TrainTime_s"], 2),
                "XGB_TrainT_s": round(r["XGB_TrainTime_s"], 2),
            })
        print(pd.DataFrame(fs_rows).to_string(index=False))

    # ── 14. EXPORT ALL RESULTS ──────────────────────────────
    print_section("EXPORTING RESULTS")
    model_summary.to_csv("/content/model_summary.csv",   index=False)
    agentic_summary.to_csv("/content/agentic_summary.csv", index=False)
    latency_df.to_csv("/content/latency_summary.csv",    index=False)
    if feature_subset_results:
        pd.DataFrame(fs_rows).to_csv("/content/feature_subset_summary.csv",
index=False)

    print("\n" + "=" * 80)
    print("✅ PIPELINE COMPLETE!")
    print("=" * 80)
    print("📊 All figures saved to /content/ directory")
    print("📈 Summary tables exported to CSV")
    print("\nOutputs generated:")
    print("  - model_performance.png")
    print("  - confusion_matrices.png")
    print("  - roc_curves.png")
    print("  - feature_importance.png")
    print("  - cv_results.png")
    print("  - latency_profile.png")
    print("  - feature_subset_experiment.png")
```

```
    print("  - noise_perturbation.png (+ per_feature variant)")
    print("  - adversarial_evasion.png (+ per_feature variant)")
    print("  - shap_summary_rf.png, shap_bar_rf.png")
    print("  - shap_summary_xgb.png, shap_bar_xgb.png")
    print("  - agentic_decisions.png")
    print("  - decision_boundary.png")
    print("  - severity_distribution.png")
    print("  - incident_precision.png")
    print("  - model_summary.csv")
    print("  - agentic_summary.csv")
    print("  - latency_summary.csv")
    print("  - feature_subset_summary.csv")
    print("=" * 80)


if __name__ == "__main__":
    main()
```

```
====================================================================
===========
LOADING AND PREPROCESSING DATA (500k samples)
====================================================================
===========
Loading: /content/Benign-Monday-no-metadata.parquet
Loading: /content/Botnet-Friday-no-metadata.parquet
Loading: /content/Bruteforce-Tuesday-no-metadata.parquet
Loading: /content/DDoS-Friday-no-metadata.parquet
Loading: /content/DoS-Wednesday-no-metadata.parquet
Loading: /content/Infiltration-Thursday-no-metadata.parquet
Loading: /content/Portscan-Friday-no-metadata.parquet
Loading: /content/WebAttacks-Thursday-no-metadata.parquet

====================================================================
===========
DATASET OVERVIEW
====================================================================
===========
Rows after preprocessing : 500000

Label distribution:
Label
BENIGN                        427147
DoS Hulk                       37683
DDoS                           27548
DoS GoldenEye                   2197
FTP-Patator                     1273
DoS slowloris                   1140
DoS Slowhttptest                1127
SSH-Patator                      726
PortScan                         400
Bot                              308
Web Attack  Brute Force         308
```

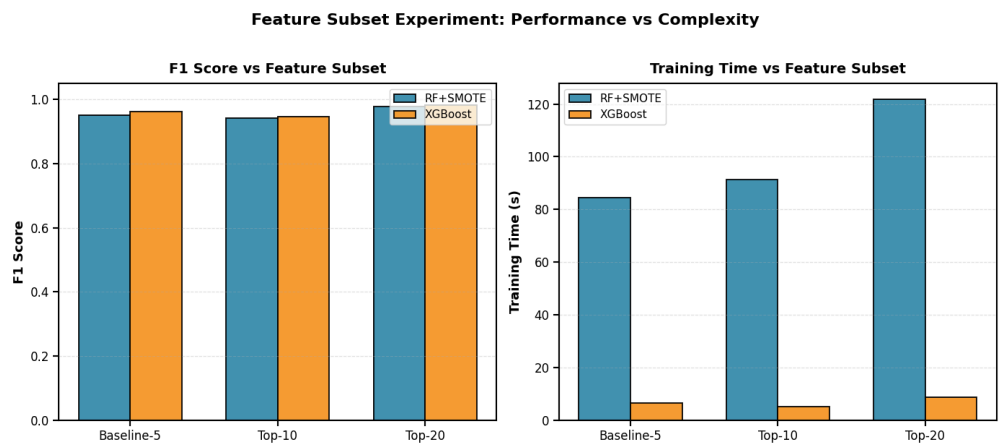
```
Web Attack  XSS                 131
Infiltration                     7
Web Attack  Sql Injection         3
Heartbleed                       2

Sample feature statistics:
                               mean           std         min
max
Flow Duration           1.807203e+07  3.639376e+07       -13.0
1.200000e+08
Total Fwd Packets       1.244162e+01  9.746904e+02         1.0
2.186580e+05
Total Backward Packets  1.425838e+01  1.280534e+03         0.0
2.912600e+05
Flow Bytes/s            4.909144e+05  7.630729e+06 -12000000.0
2.070000e+09
Flow Packets/s          2.013056e+04  1.214563e+05  -2000000.0
4.000000e+06

======================================================================
===========
TRAINING MODELS
======================================================================
===========
Building IsolationForest...
Building RF + SMOTE...
Building XGBoost + SMOTE...

======================================================================
===========
BATCH INFERENCE LATENCY MEASUREMENT
======================================================================
===========

======================================================================
===========
LATENCY REPORT
======================================================================
===========
                  Operation  Mean (ms)  Std (ms)  Min (ms)  Max (ms)
Total (s)  N
                IF Training   4055.553       0.0  4055.553  4055.553
4.0556  1
                RF Training  81402.403       0.0 81402.403 81402.403
81.4024  1
               XGB Training   4993.968       0.0  4993.968  4993.968
4.9940  1
    RF Batch Inference (n=1)     28.560       0.0    28.560    28.560
0.0286  1
   XGB Batch Inference (n=1)     21.200       0.0    21.200    21.200
0.0212  1
    IF Batch Inference (n=1)     26.931       0.0    26.931    26.931
0.0269  1
```

```
   RF Batch Inference (n=10)     25.727       0.0    25.727    25.727
0.0257  1
  XGB Batch Inference (n=10)     14.407       0.0    14.407    14.407
0.0144  1
   IF Batch Inference (n=10)     26.901       0.0    26.901    26.901
0.0269  1
  RF Batch Inference (n=100)     25.686       0.0    25.686    25.686
0.0257  1
 XGB Batch Inference (n=100)     10.302       0.0    10.302    10.302
0.0103  1
  IF Batch Inference (n=100)     31.582       0.0    31.582    31.582
0.0316  1
 RF Batch Inference (n=1000)     35.686       0.0    35.686    35.686
0.0357  1
XGB Batch Inference (n=1000)     13.537       0.0    13.537    13.537
0.0135  1
 IF Batch Inference (n=1000)     41.882       0.0    41.882    41.882
0.0419  1

======================================================================
===========
CROSS-VALIDATION (3-FOLD)
======================================================================
===========

======================================================================
===========
CROSS-VALIDATION — Random Forest + SMOTE
======================================================================
===========
Random Forest + SMOTE CV F1 Scores : [0.94559023 0.94648558
0.95060189]
Random Forest + SMOTE CV Mean F1   : 0.9476 (+/- 0.0044)

======================================================================
===========
CROSS-VALIDATION — Isolation Forest
======================================================================
===========
Isolation Forest CV F1 Scores : [nan nan nan]
Isolation Forest CV Mean F1   : nan (+/- nan)

======================================================================
===========
CROSS-VALIDATION — XGBoost
======================================================================
===========
XGBoost CV F1 Scores : [0.9607146  0.96298877 0.96563125]
XGBoost CV Mean F1   : 0.9631 (+/- 0.0040)

======================================================================
===========
MODEL EVALUATION ON HOLD-OUT TEST SET
```

```
======================================================================
===========

======================================================================
===========
Random Forest + SMOTE RESULTS
======================================================================
===========
Accuracy : 0.985460
Precision: 0.939233
Recall   : 0.962482
F1-score : 0.950715

Confusion Matrix:
[[126783   1361]
 [   820  21036]]

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99    128144
           1       0.94      0.96      0.95     21856

    accuracy                           0.99    150000
   macro avg       0.97      0.98      0.97    150000
weighted avg       0.99      0.99      0.99    150000


======================================================================
===========
Isolation Forest RESULTS
======================================================================
===========
Accuracy : 0.720893
Precision: 0.049243
Recall   : 0.050009
F1-score : 0.049623

Confusion Matrix:
[[107041  21103]
 [ 20763   1093]]

Classification Report:
              precision    recall  f1-score   support

           0       0.84      0.84      0.84    128144
           1       0.05      0.05      0.05     21856

    accuracy                           0.72    150000
   macro avg       0.44      0.44      0.44    150000
weighted avg       0.72      0.72      0.72    150000
```

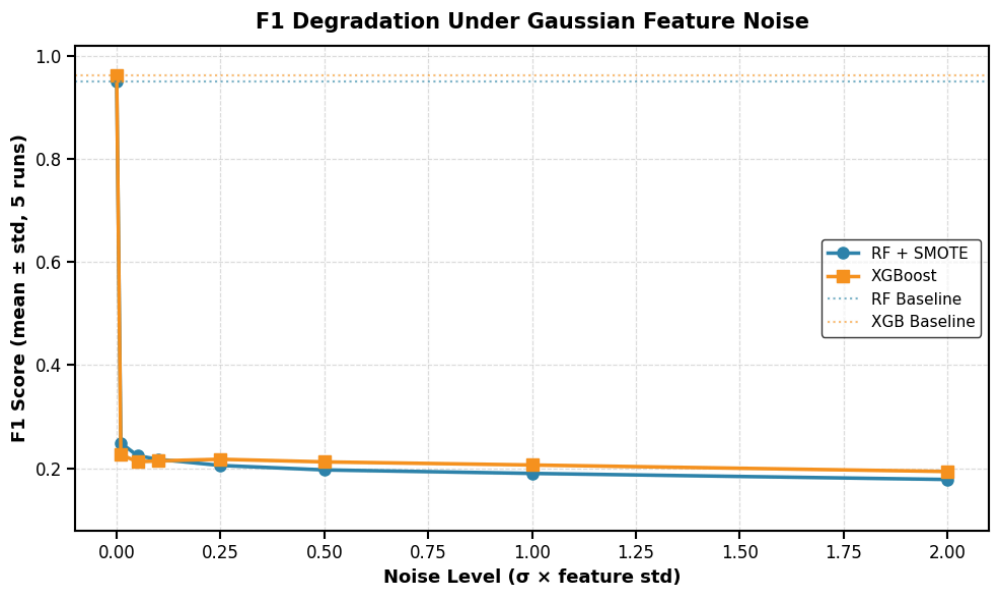
```
======================================================================
===========
XGBoost RESULTS
======================================================================
===========
Accuracy : 0.988660
Precision: 0.946717
Recall   : 0.977169
F1-score : 0.961702

Confusion Matrix:
[[126942   1202]
 [   499  21357]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      0.99    128144
           1       0.95      0.98      0.96     21856

    accuracy                           0.99    150000
   macro avg       0.97      0.98      0.98    150000
weighted avg       0.99      0.99      0.99    150000


======================================================================
===========
FEATURE SUBSET EXPERIMENT: 5 vs TOP-10 vs TOP-20
======================================================================
===========
  Features available in dataset : 43

  Top-20 features by Gini importance:
     1. Bwd Packet Length Std                          0.122260
     2. Packet Length Variance                         0.098845
     3. Bwd Packet Length Mean                         0.088593
     4. Packet Length Std                              0.082053
     5. Bwd Packet Length Max                          0.078469
     6. Avg Bwd Segment Size                           0.068228
     7. Packet Length Max                              0.040011
     8. Fwd IAT Mean                                   0.034374
     9. Bwd Packets/s                                  0.032372
    10. Total Backward Packets                         0.025916
    11. Subflow Fwd Packets                            0.025652
    12. Packet Length Mean                             0.023721
    13. Subflow Bwd Bytes                              0.022620
    14. Fwd Packet Length Max                          0.021701
    15. Total Fwd Packets                              0.020634
    16. Fwd IAT Std                                    0.020605
    17. Fwd Packet Length Mean                         0.018508
    18. Fwd IAT Total                                  0.017358
    19. Subflow Bwd Packets                            0.015797
    20. Flow IAT Std                                   0.015157
```

```
  ── Baseline-5 (5 features) ──
    RF  — F1=0.9507  Acc=0.9855  TrainT=84.47s  InfT=0.0043ms
    XGB — F1=0.9617  Acc=0.9887  TrainT=6.58s  InfT=0.0019ms

  ── Top-10 (10 features) ──
    RF  — F1=0.9426  Acc=0.9828  TrainT=91.44s  InfT=0.0058ms
    XGB — F1=0.9465  Acc=0.9839  TrainT=5.07s  InfT=0.0020ms

  ── Top-20 (20 features) ──
    RF  — F1=0.9778  Acc=0.9935  TrainT=121.77s  InfT=0.0047ms
    XGB — F1=0.9817  Acc=0.9946  TrainT=8.68s  InfT=0.0019ms

======================================================================
===========
FEATURE SUBSET EXPERIMENT — SUMMARY TABLE
======================================================================
===========
    Subset  N_feat  RF_F1  RF_Acc  XGB_F1  XGB_Acc
Baseline-5       5 0.9507  0.9855  0.9617   0.9887
    Top-10      10 0.9426  0.9828  0.9465   0.9839
    Top-20      20 0.9778  0.9935  0.9817   0.9946
```

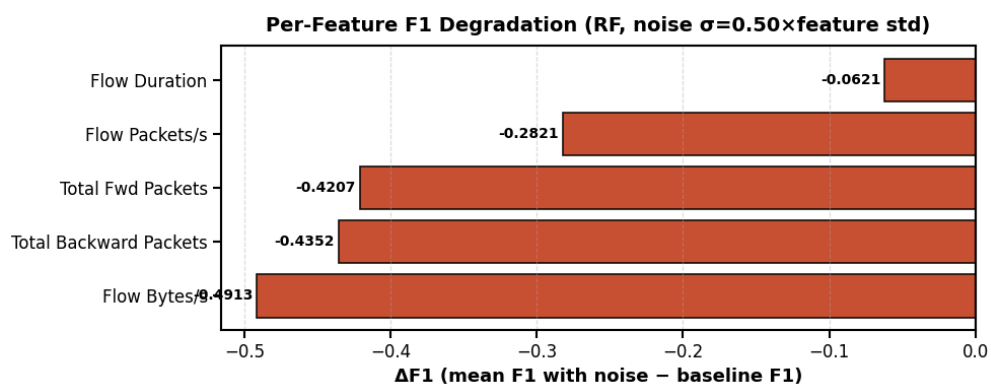



```
======================================================================
===========
GAUSSIAN NOISE PERTURBATION TEST — F1 DEGRADATION
======================================================================
===========
  Baseline F1  RF=0.9507   XGB=0.9617

   Noise Level   RF F1 mean  RF F1 std   XGB F1 mean XGB F1 std
RF Δ    XGB Δ
  ------------  ----------- ----------  ------------ ----------  ----
---- --------
          0.00       0.9507     0.0000        0.9617     0.0000
+0.0000  +0.0000
          0.01       0.2488     0.0023        0.2263     0.0028
-0.7019  -0.7354
          0.05       0.2243     0.0013        0.2127     0.0022
```

```
-0.7264  -0.7490
          0.10       0.2172     0.0014        0.2142     0.0026
-0.7335  -0.7475
          0.25       0.2054     0.0015        0.2174     0.0017
-0.7454  -0.7443
          0.50       0.1967     0.0018        0.2123     0.0016
-0.7540  -0.7494
          1.00       0.1899     0.0015        0.2062     0.0016
-0.7608  -0.7555
          2.00       0.1781     0.0014        0.1936     0.0009
-0.7726  -0.7681
```

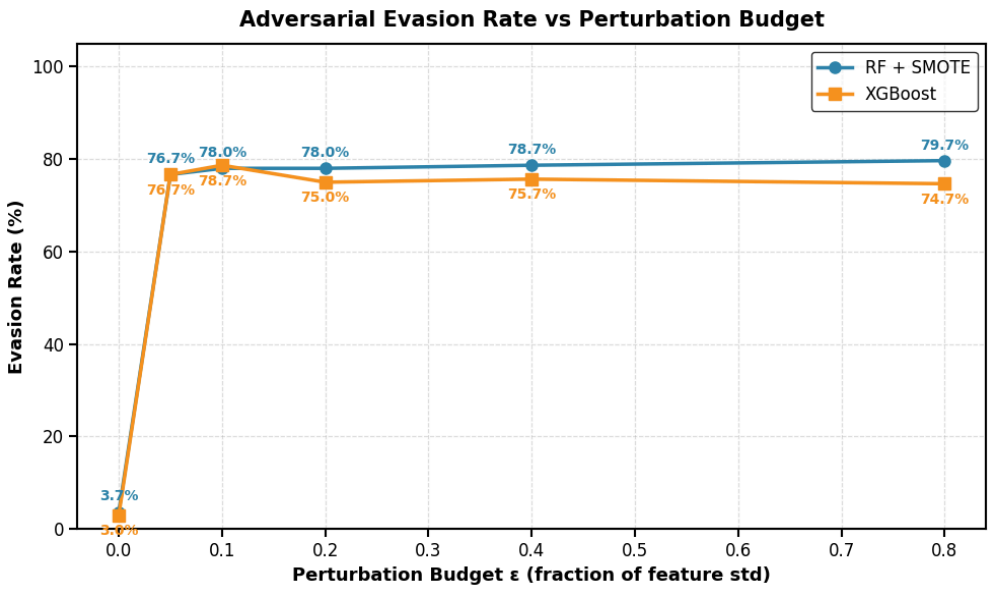



```
Per-feature F1 degradation (noise_level=0.50, RF, n=5 repeats):
  Flow Duration                          F1=0.8887  ΔF1=-0.0621
  Total Fwd Packets                      F1=0.5300  ΔF1=-0.4207
  Total Backward Packets                 F1=0.5155  ΔF1=-0.4352
  Flow Bytes/s                           F1=0.4594  ΔF1=-0.4913
  Flow Packets/s                         F1=0.6686  ΔF1=-0.2821
```

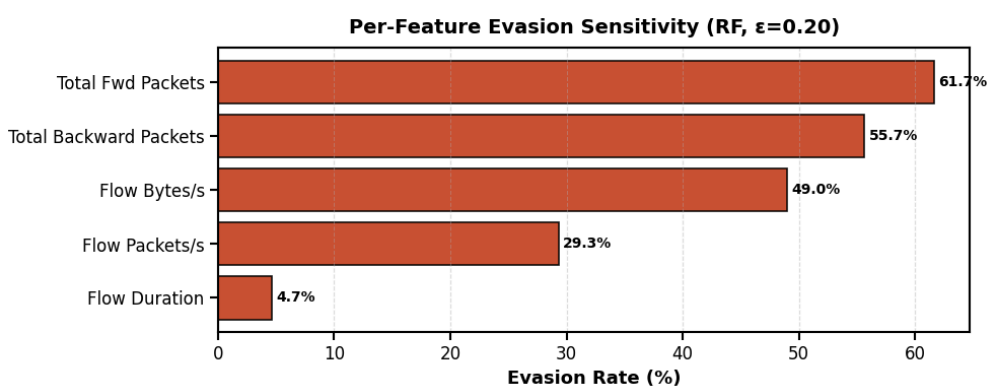



```
=================================================================
===========
ADVERSARIAL EVASION TEST
```

```
======================================================================
===========
  Attack samples selected for evasion test: 300

   Epsilon   RF Evasion Rate   XGB Evasion Rate
  --------  ----------------  -----------------
      0.00           3.6667%            3.0000%
      0.05          76.6667%           76.6667%
      0.10          78.0000%           78.6667%
      0.20          78.0000%           75.0000%
      0.40          78.6667%           75.6667%
      0.80          79.6667%           74.6667%
```

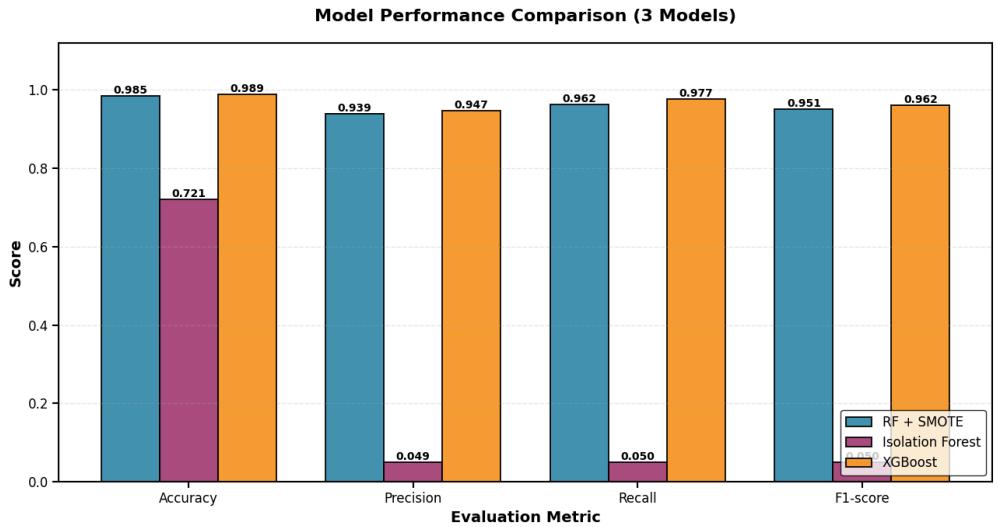


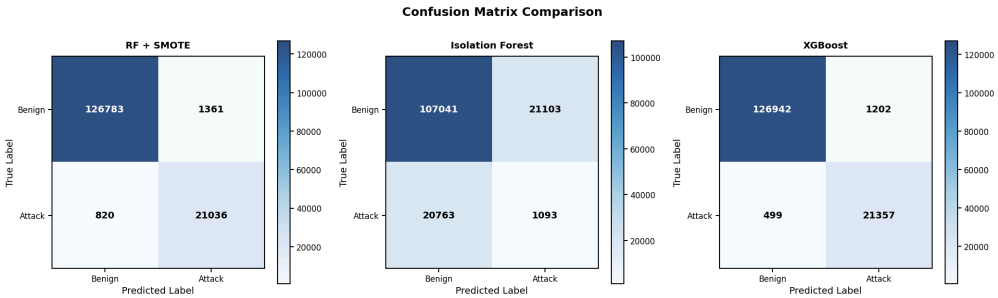

```
  Per-feature evasion sensitivity (epsilon=0.20, RF):
    Flow Duration                              evasion=4.6667%
    Total Fwd Packets                          evasion=61.6667%
    Total Backward Packets                     evasion=55.6667%
    Flow Bytes/s                               evasion=49.0000%
    Flow Packets/s                             evasion=29.3333%
```

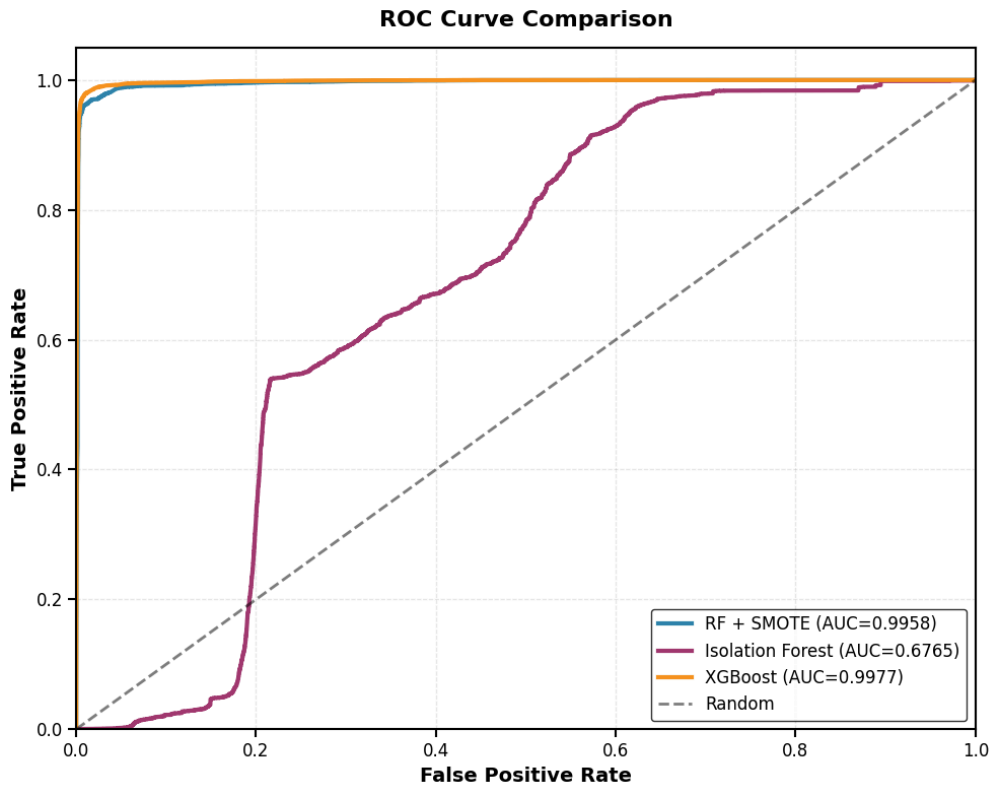


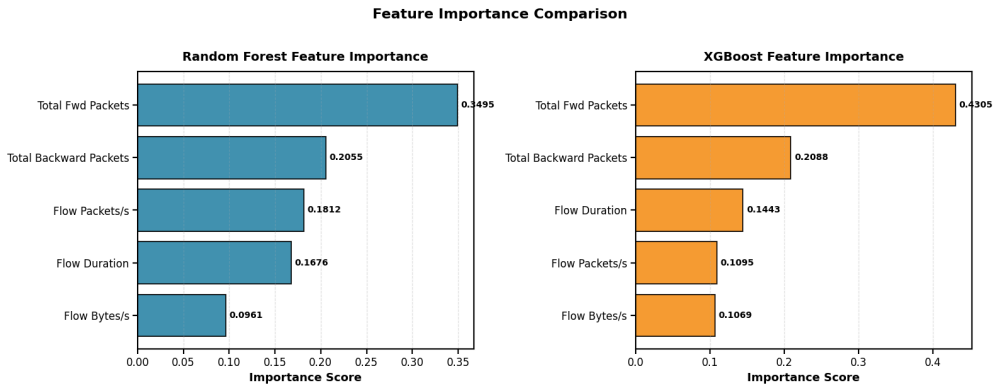

```
======================================================================
===========
```

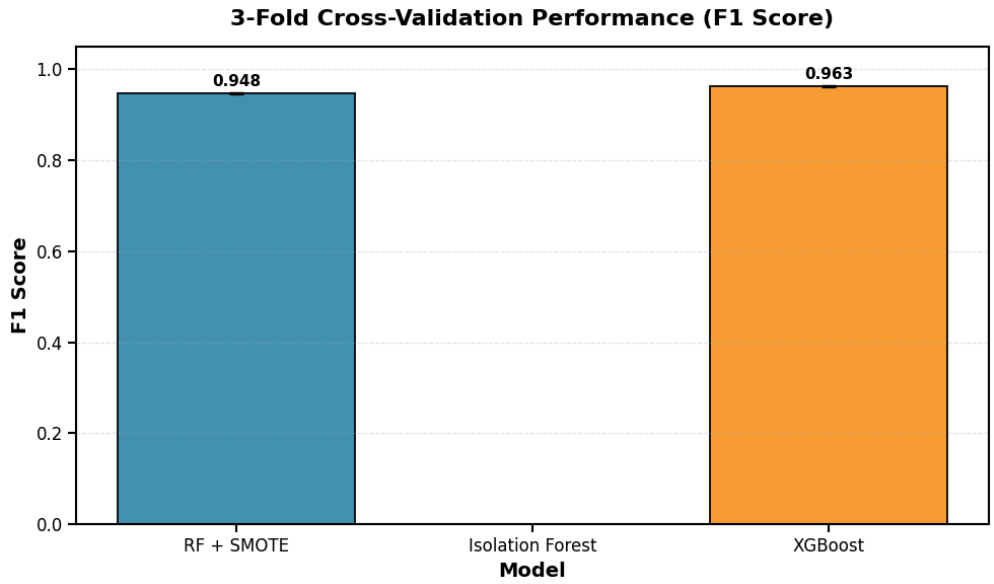
```
GENERATING PUBLICATION-QUALITY FIGURES
======================================================================
===========
```

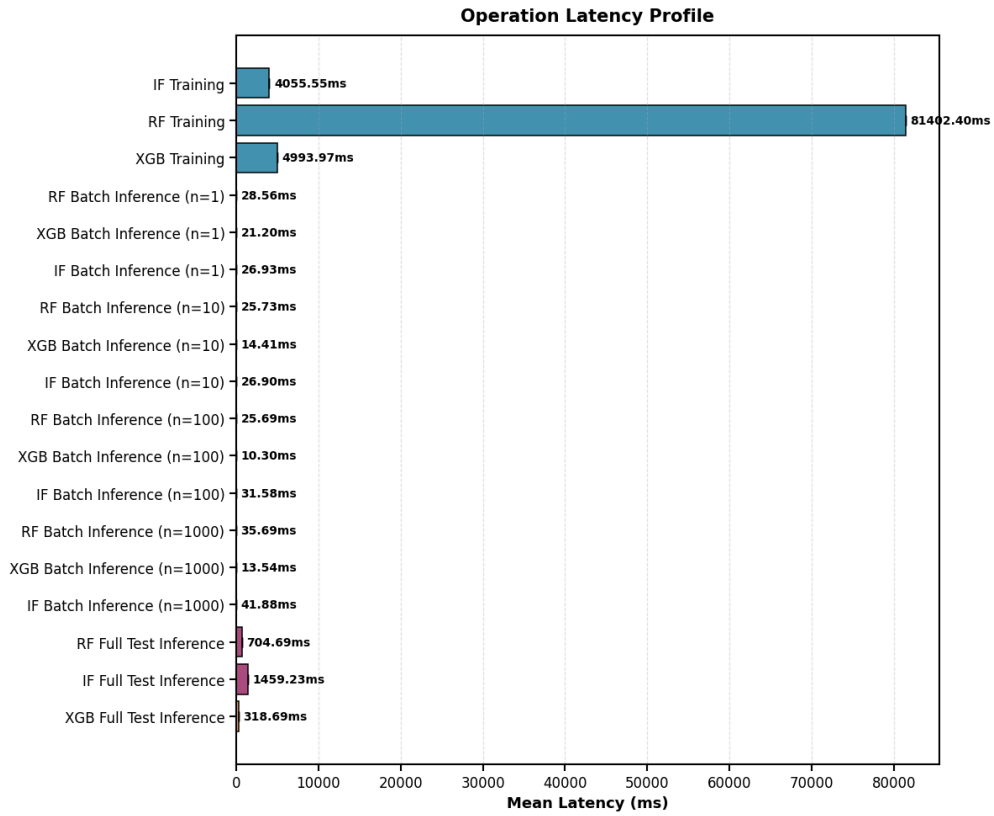




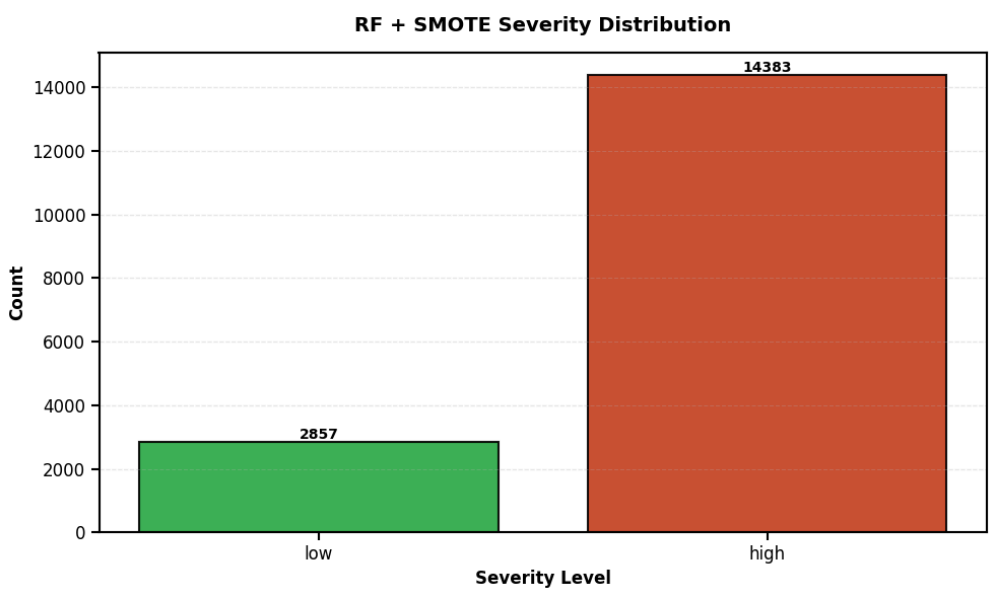


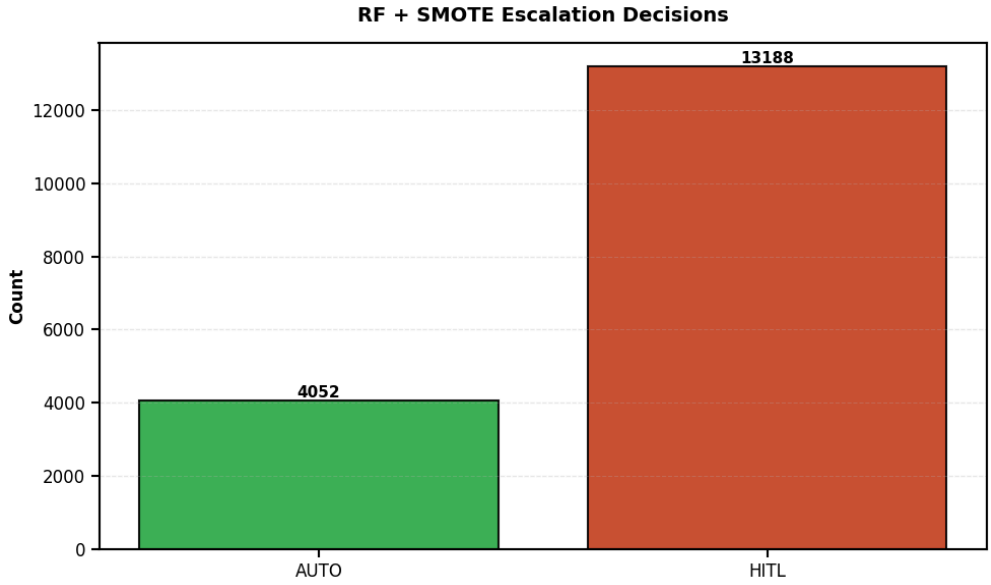

Feature Importance Comparison
Random Forest Feature Importance
Total Fwd Packets
Total Backward Packets
Flow Packets/s
Flow Duration
Flow Bytes/s
0.3495
0.2055
0.1812
0.1676
0.0961
0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35
Importance Score
XGBoost Feature Importance
Total Fwd Packets
Total Backward Packets
Flow Duration
Flow Packets/s
Flow Bytes/s
0.4305
0.2088
0.1443
0.1095
0.1069
0.0 0.1 0.2 0.3 0.4
Importance Score

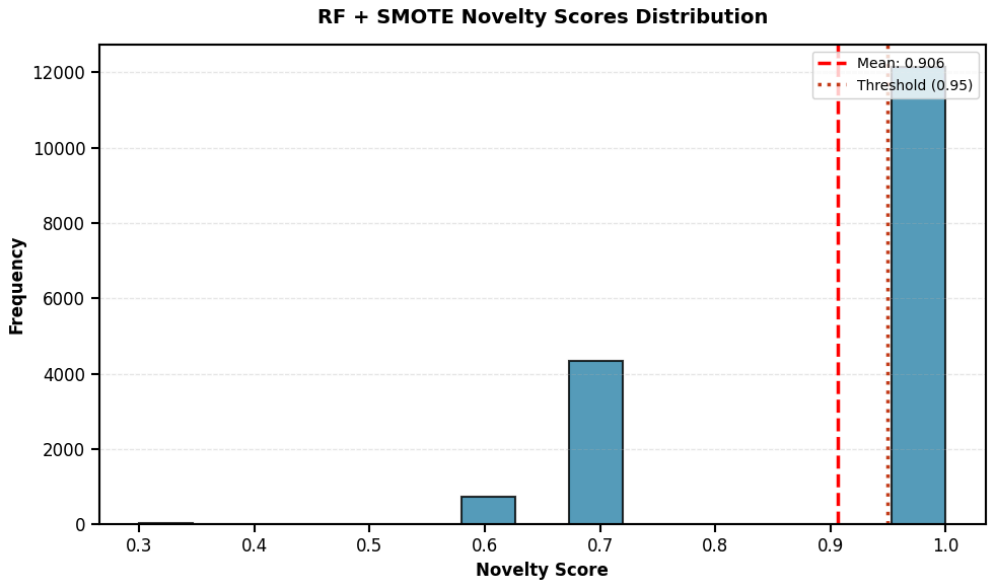

3-Fold Cross-Validation Performance (F1 Score)
F1 Score
0.0 0.2 0.4 0.6 0.8 1.0
0.948
0.963
RF + SMOTE
Isolation Forest
XGBoost
Model


Operation Latency Profile
IF Training 4055.55ms
RF Training 81402.40ms
XGB Training 4993.97ms
RF Batch Inference (n=1) 28.56ms
XGB Batch Inference (n=1) 21.20ms
IF Batch Inference (n=1) 26.93ms
RF Batch Inference (n=10) 25.73ms
XGB Batch Inference (n=10) 14.41ms
IF Batch Inference (n=10) 26.90ms
RF Batch Inference (n=100) 25.69ms
XGB Batch Inference (n=100) 10.30ms
IF Batch Inference (n=100) 31.58ms
RF Batch Inference (n=1000) 35.69ms
XGB Batch Inference (n=1000) 13.54ms
IF Batch Inference (n=1000) 41.88ms
RF Full Test Inference 704.69ms
IF Full Test Inference 1459.23ms
XGB Full Test Inference 318.69ms
0 10000 20000 30000 40000 50000 60000 70000 80000
Mean Latency (ms)

```
======================================================================
===========
AGENTIC PIPELINE RUN — RANDOM FOREST + SMOTE
======================================================================
===========

======================================================================
===========
AGENTIC SOC RESULTS — RF + SMOTE
======================================================================
===========
Total Logs     : 150000
Anomalies      : 22397
Incidents      : 17240
AUTO Decisions : 4052
HITL Decisions : 13188
Incident TP    : 16021
Incident FP    : 1219

Severity Distribution:
  high       : 14383
  low        : 2857

Novelty Summary:
  Avg Novelty  : 0.9063
  Max Novelty  : 1.0000
  Min Novelty  : 0.3000

Confidence Summary:
  Avg Confidence: 0.9461
  Max Confidence: 0.9977
  Min Confidence: 0.5002

HITL samples for SHAP: 13515
```

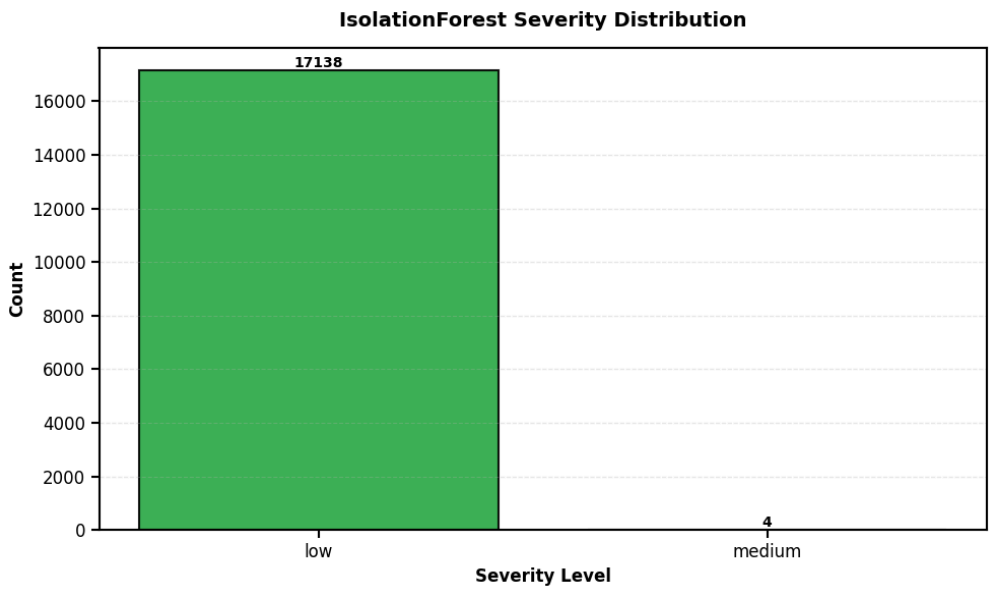


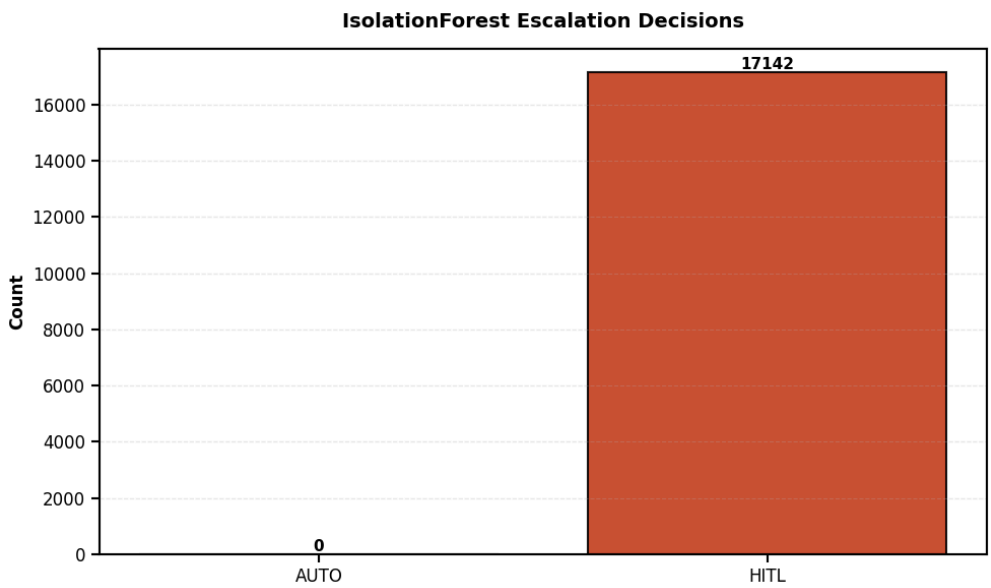


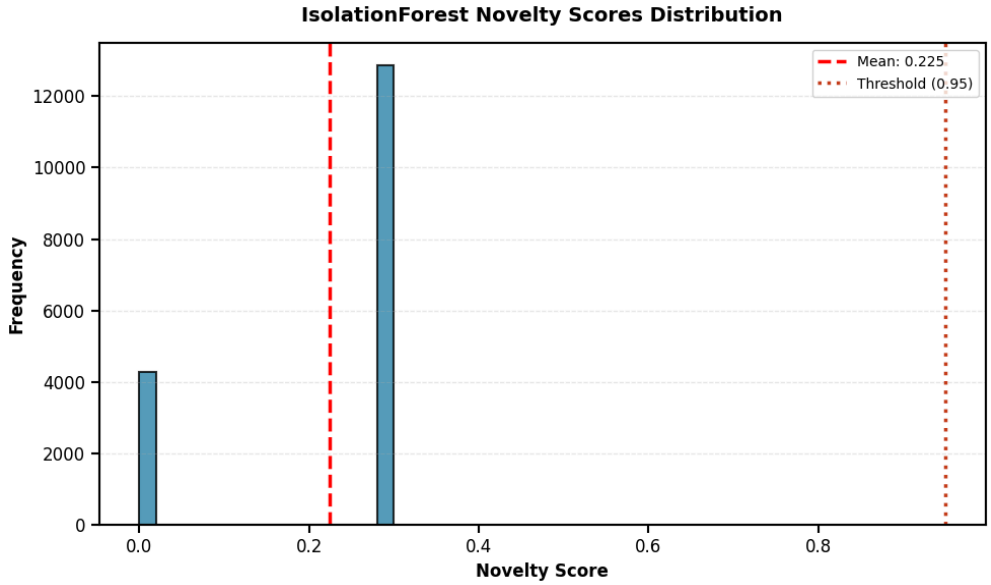



```
======================================================================
===========
AGENTIC PIPELINE RUN — ISOLATION FOREST
======================================================================
===========

======================================================================
===========
AGENTIC SOC RESULTS — IsolationForest
======================================================================
===========
Total Logs     : 150000
Anomalies      : 22196
Incidents      : 17142
AUTO Decisions : 0
HITL Decisions : 17142
Incident TP    : 664
```

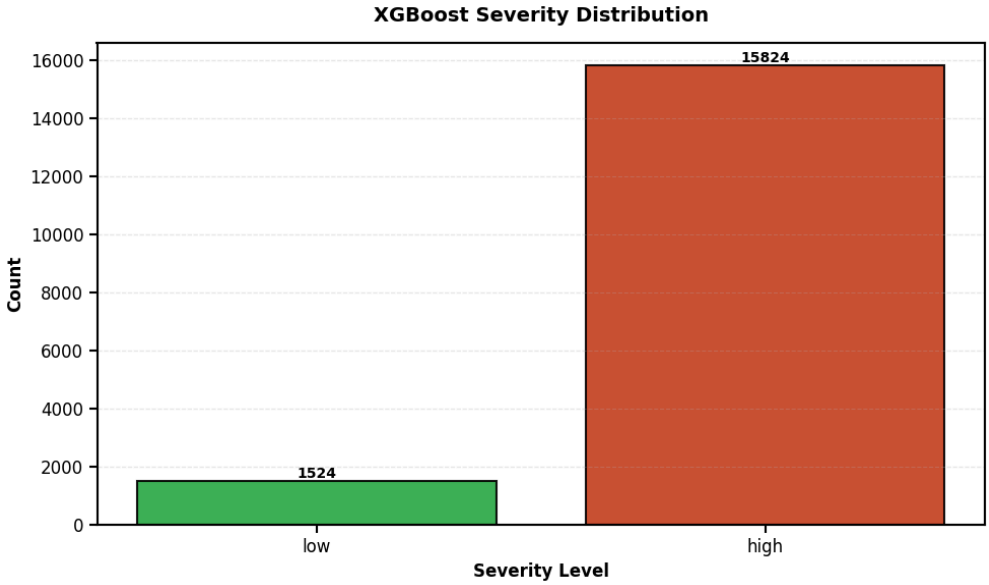
```
Incident FP    : 16478

Severity Distribution:
  low        : 17138
  medium     : 4

Novelty Summary:
  Avg Novelty  : 0.2249
  Max Novelty  : 0.3000
  Min Novelty  : 0.0000

Confidence Summary:
  Avg Confidence: 0.4785
  Max Confidence: 0.5000
  Min Confidence: 0.4064

HITL samples for SHAP: 22196
```

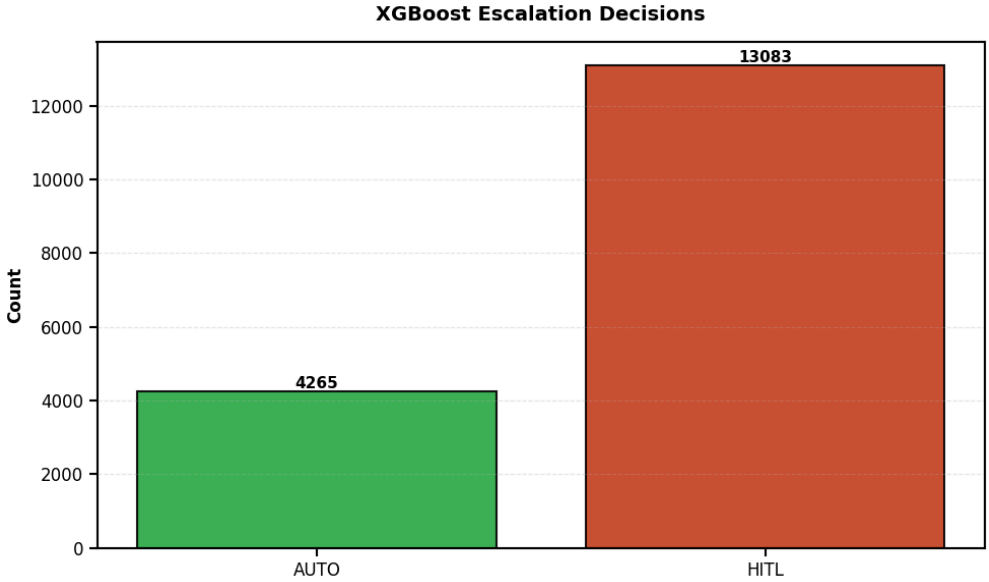


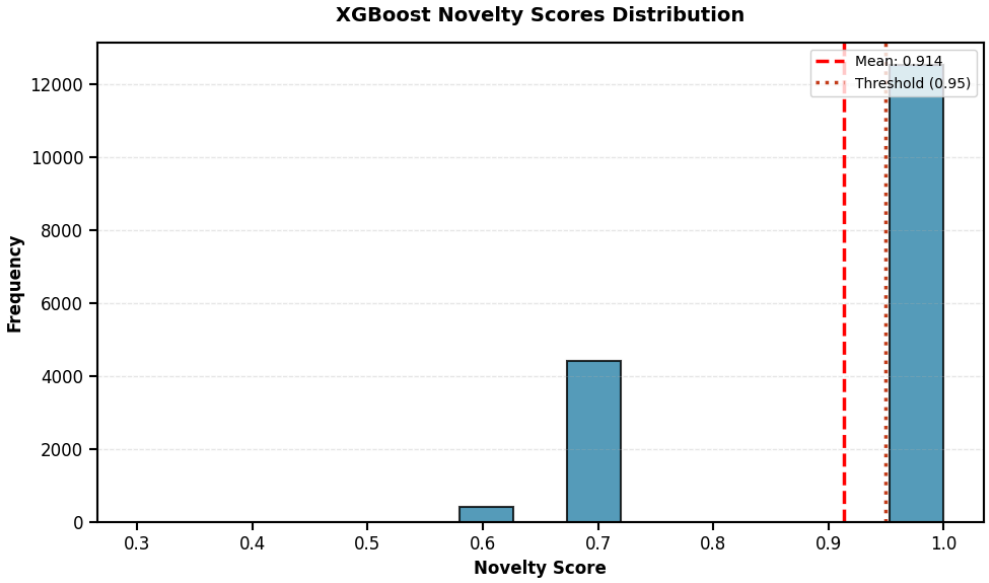




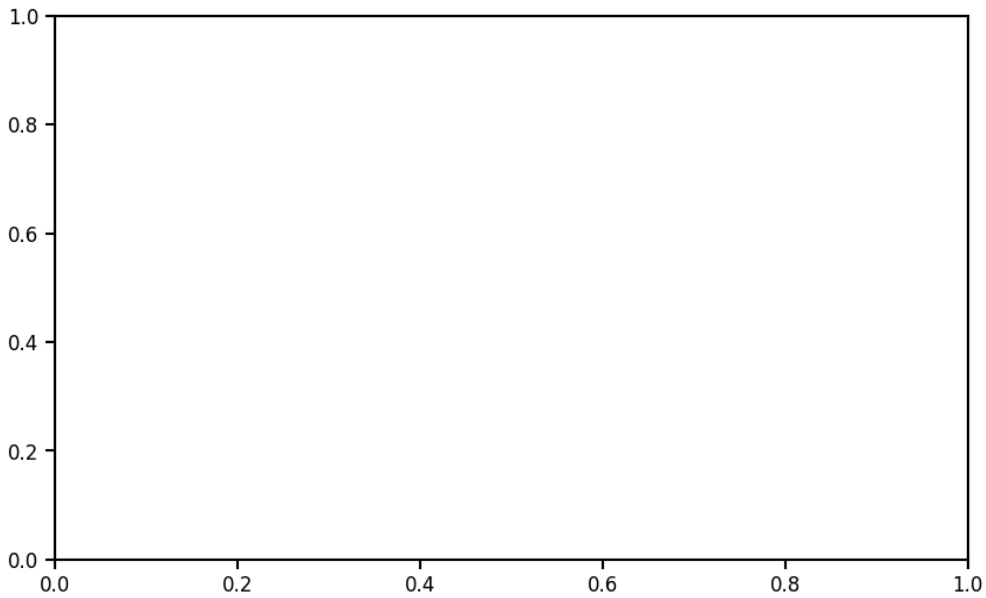

```
=================================================================
===========
AGENTIC PIPELINE RUN — XGBOOST
=================================================================
===========

=================================================================
===========
AGENTIC SOC RESULTS — XGBoost
=================================================================
===========
Total Logs     : 150000
Anomalies      : 22559
Incidents      : 17348
AUTO Decisions : 4265
HITL Decisions : 13083
Incident TP    : 16274
Incident FP    : 1074

Severity Distribution:
  high       : 15824
  low        : 1524

Novelty Summary:
  Avg Novelty  : 0.9136
  Max Novelty  : 1.0000
  Min Novelty  : 0.3000

Confidence Summary:
  Avg Confidence: 0.9704
  Max Confidence: 0.9992
  Min Confidence: 0.5003

HITL samples for SHAP: 13235
```

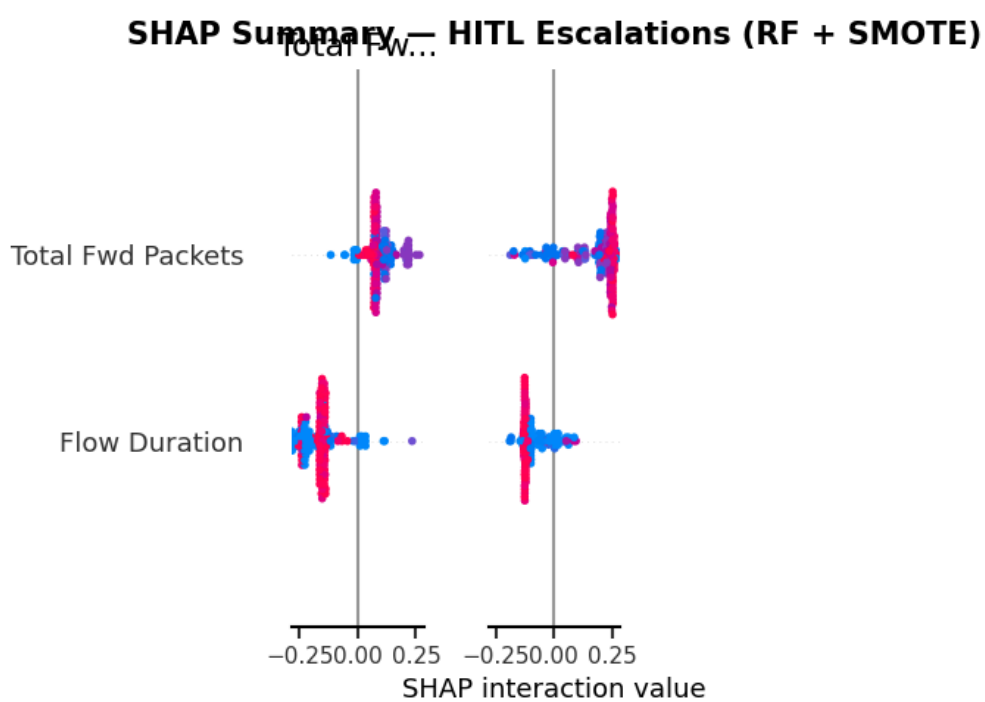

XGBoost Severity Distribution
Count
Severity Level
low
high
1524
15824
0
2000
4000
6000
8000
10000
12000
14000
16000


XGBoost Escalation Decisions
Count
AUTO
HITL
4265
13083
0
2000
4000
6000
8000
10000
12000

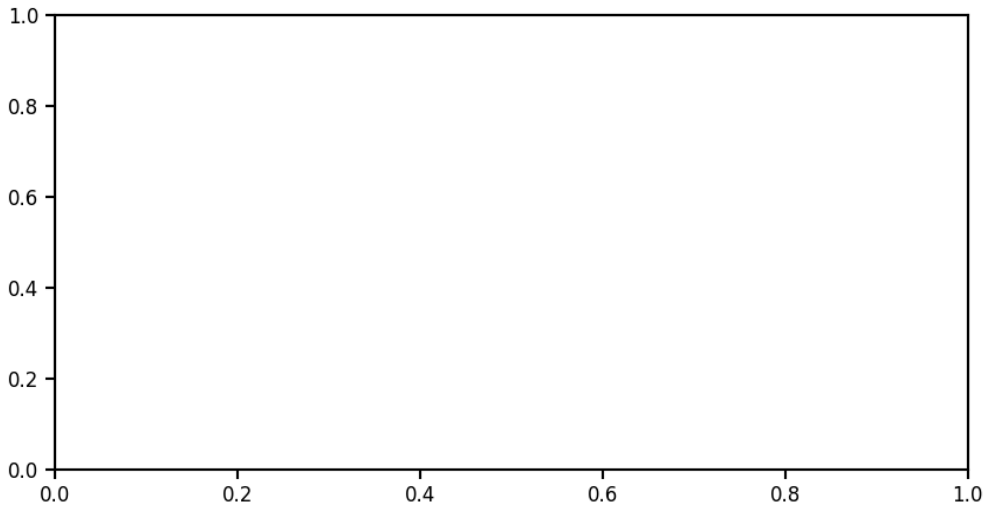

XGBoost Novelty Scores Distribution
Mean: 0.914
Threshold (0.95)
Frequency
Novelty Score
0
2000
4000
6000
8000
10000
12000
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0

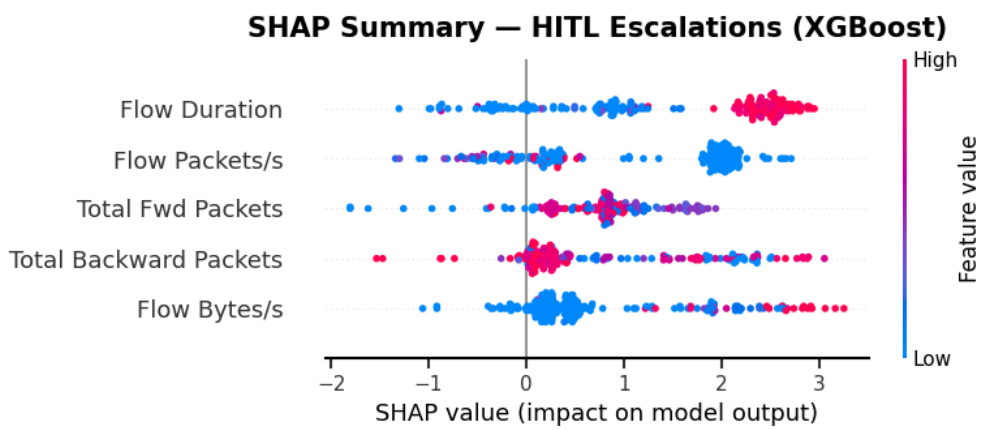
```
======================================================================
===========
SHAP EXPLANATIONS FOR HITL ESCALATIONS
======================================================================
===========

======================================================================
===========
SHAP EXPLANATIONS FOR HITL ESCALATIONS — RF + SMOTE
======================================================================
===========
  HITL samples explained : 200
  Feature count          : 5
```

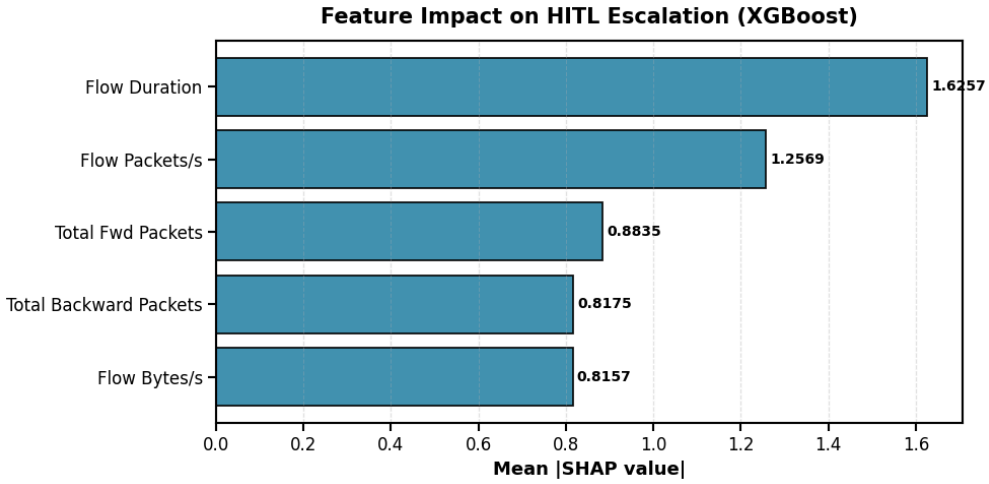




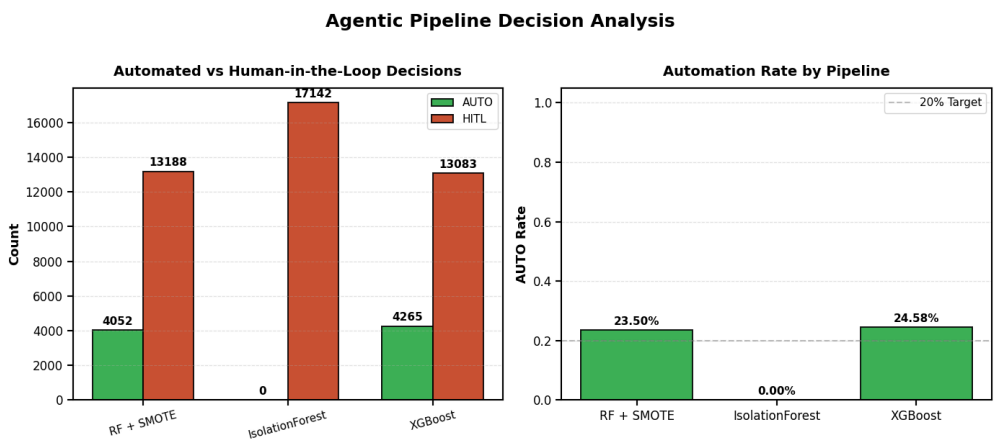
```
  SHAP computation failed: shape mismatch: objects cannot be
broadcast to a single shape.  Mismatch is between arg 2 with shape
(5, 2, 2) and arg 3 with shape (5,).

======================================================================
===========
SHAP EXPLANATIONS FOR HITL ESCALATIONS — XGBoost
======================================================================
===========
  HITL samples explained : 200
  Feature count          : 5
```

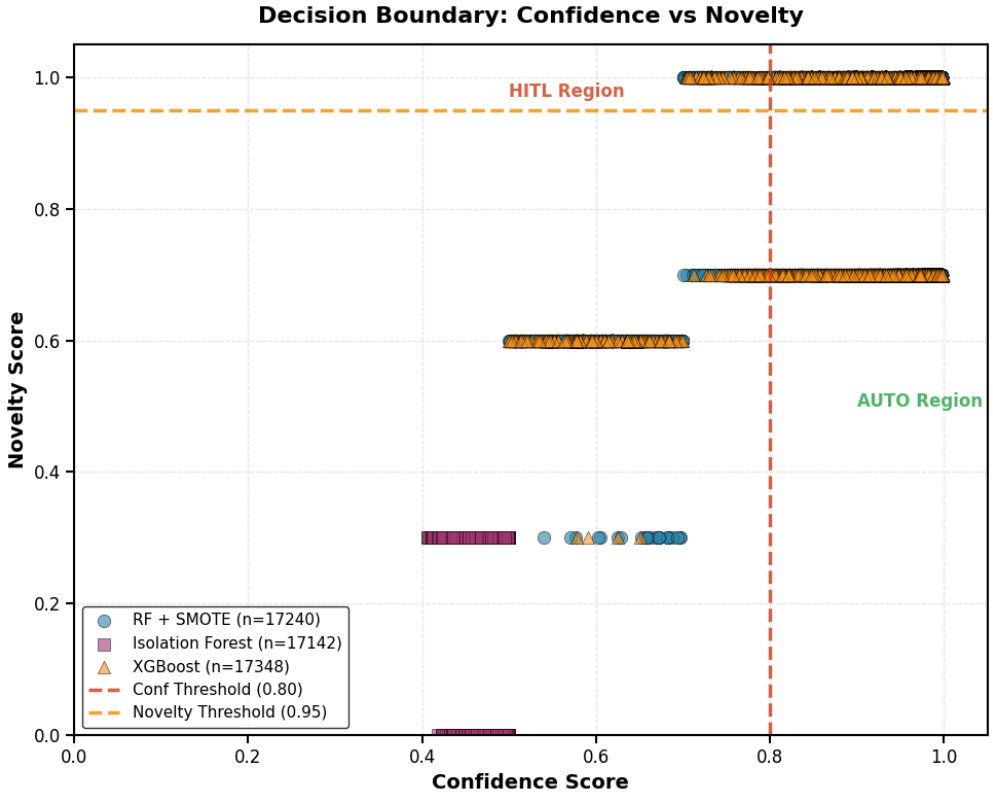


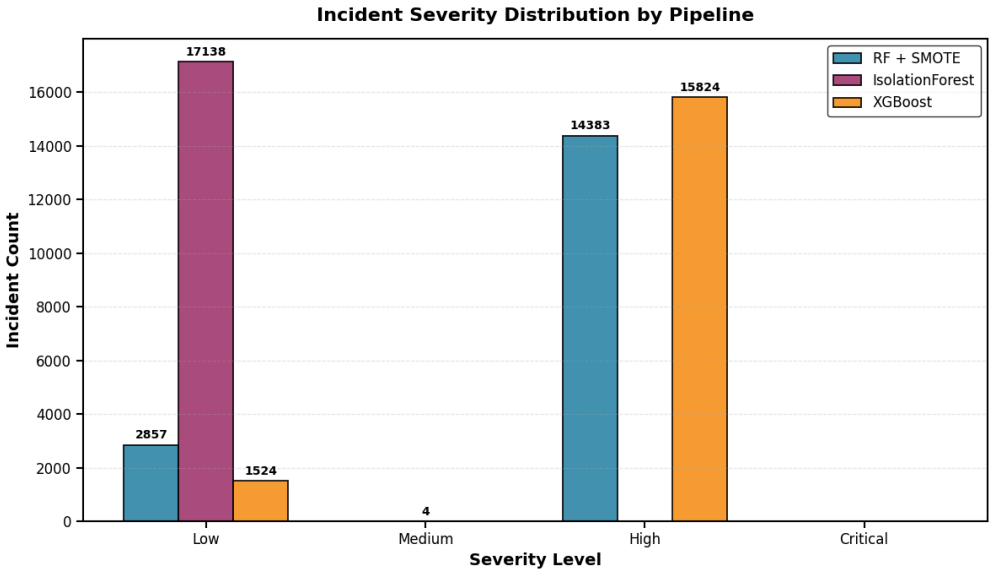


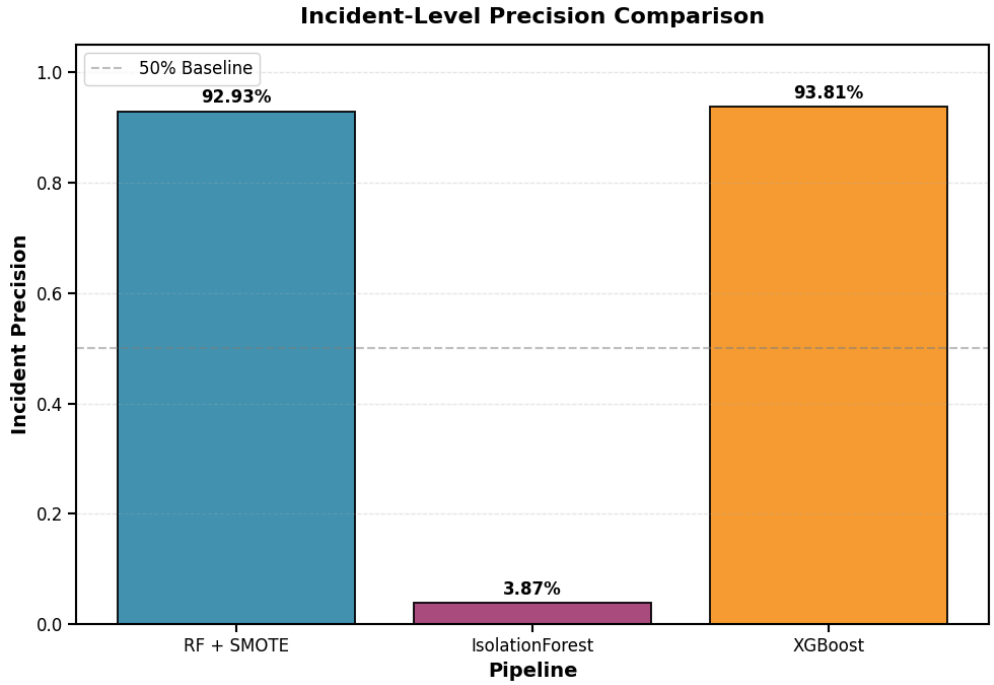



```
  Top-3 SHAP drivers for HITL escalations:
    1. Flow Duration                        mean|SHAP|=1.62573
    2. Flow Packets/s                       mean|SHAP|=1.25691
    3. Total Fwd Packets                    mean|SHAP|=0.88345

======================================================================
===========
GENERATING AGENTIC PIPELINE FIGURES
======================================================================
===========
```




Decision Boundary: Confidence vs Novelty
HITL Region
AUTO Region
Novelty Score
Confidence Score
RF + SMOTE (n=17240)
Isolation Forest (n=17142)
XGBoost (n=17348)
Conf Threshold (0.80)
Novelty Threshold (0.95)
0.0
0.2
0.4
0.6
0.8
1.0


Incident Severity Distribution by Pipeline
RF + SMOTE
IsolationForest
XGBoost
Incident Count
Severity Level
Low
Medium
High
Critical
2857
17138
1524
4
14383
15824
0
2000
4000
6000
8000
10000
12000
14000
16000




```
======================================================================
===========
FINAL SUMMARY TABLES
======================================================================
===========
Model Performance Summary:
           Model  Accuracy  Precision   Recall  F1-score
      RF + SMOTE  0.985460   0.939233 0.962482  0.950715
Isolation Forest  0.720893   0.049243 0.050009  0.049623
         XGBoost  0.988660   0.946717 0.977169  0.961702

Agentic Pipeline Summary:
       Pipeline   Logs  Anomalies  Incidents  AUTO  HITL  AUTO Rate
HITL Rate  Incident Precision  Avg Novelty  Max Novelty  Avg
Confidence
     RF + SMOTE 150000      22397      17240  4052 13188   0.235035
0.764965            0.929292     0.906282          1.0
0.946143
IsolationForest 150000      22196      17142     0 17142   0.000000
1.000000            0.038735     0.224851          0.3
0.478538
        XGBoost 150000      22559      17348  4265 13083   0.245850
0.754150            0.938091     0.913621          1.0
0.970385

Latency Summary:
                     Operation  Mean (ms)  Std (ms)  Min (ms)  Max
(ms)  Total (s)      N
                   IF Training   4055.553     0.000  4055.553
4055.553     4.0556      1
                   RF Training  81402.403     0.000 81402.403
81402.403    81.4024      1
```

```
                  XGB Training   4993.968     0.000  4993.968
4993.968     4.9940      1
       RF Batch Inference (n=1)     28.560     0.000    28.560
28.560     0.0286      1
      XGB Batch Inference (n=1)     21.200     0.000    21.200
21.200     0.0212      1
       IF Batch Inference (n=1)     26.931     0.000    26.931
26.931     0.0269      1
      RF Batch Inference (n=10)     25.727     0.000    25.727
25.727     0.0257      1
     XGB Batch Inference (n=10)     14.407     0.000    14.407
14.407     0.0144      1
      IF Batch Inference (n=10)     26.901     0.000    26.901
26.901     0.0269      1
     RF Batch Inference (n=100)     25.686     0.000    25.686
25.686     0.0257      1
    XGB Batch Inference (n=100)     10.302     0.000    10.302
10.302     0.0103      1
     IF Batch Inference (n=100)     31.582     0.000    31.582
31.582     0.0316      1
    RF Batch Inference (n=1000)     35.686     0.000    35.686
35.686     0.0357      1
   XGB Batch Inference (n=1000)     13.537     0.000    13.537
13.537     0.0135      1
    IF Batch Inference (n=1000)     41.882     0.000    41.882
41.882     0.0419      1
         RF Full Test Inference    704.687     0.000   704.687
704.687     0.7047      1
         IF Full Test Inference   1459.233     0.000  1459.233
1459.233     1.4592      1
        XGB Full Test Inference    318.688     0.000   318.688
318.688     0.3187      1
                 Inference (rf)     31.034     5.662    23.574
414.814  4655.1541 150000
            Inference (iforest)     34.826     8.693    28.136
353.080  5223.8835 150000
                Inference (xgb)      1.992     2.378     1.438
565.823   298.8172 150000
SHAP TreeExplainer (RF + SMOTE)   1149.822     0.000  1149.822
1149.822     1.1498      1
   SHAP TreeExplainer (XGBoost)    299.490     0.000   299.490
299.490     0.2995      1

Feature Subset Experiment Summary:
    Subset  N_feat  RF_F1  RF_Acc  XGB_F1  XGB_Acc  RF_TrainT_s
XGB_TrainT_s
Baseline-5       5 0.9507  0.9855  0.9617   0.9887        84.47
6.58
    Top-10      10 0.9426  0.9828  0.9465   0.9839        91.44
5.07
    Top-20      20 0.9778  0.9935  0.9817   0.9946       121.77
8.68
```

```
======================================================================
EXPORTING RESULTS
======================================================================

======================================================================
✅ PIPELINE COMPLETE!
======================================================================
📊 All figures saved to /content/ directory
📈 Summary tables exported to CSV

Outputs generated:
  - model_performance.png
  - confusion_matrices.png
  - roc_curves.png
  - feature_importance.png
  - cv_results.png
  - latency_profile.png
  - feature_subset_experiment.png
  - noise_perturbation.png (+ per_feature variant)
  - adversarial_evasion.png (+ per_feature variant)
  - shap_summary_rf.png, shap_bar_rf.png
  - shap_summary_xgb.png, shap_bar_xgb.png
  - agentic_decisions.png
  - decision_boundary.png
  - severity_distribution.png
  - incident_precision.png
  - model_summary.csv
  - agentic_summary.csv
  - latency_summary.csv
  - feature_subset_summary.csv
======================================================================
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.